



Ministry of Higher Education
and Scientific Research
*** * ***

University of Carthage
*** * ***

National Institute
of Applied Science and
Technology



PROJET PERSONNEL PROFESSIONNEL

SPECIALITY: SOFTWARE ENGINEERING

LEVEL: 3RD YEAR ENGINEERING

Topic:

**Design and Software Architecture of an 18-DOF
Humanoid Robot: ROS 2 Control, Simulation and an
LLM-Based Natural-Language Command Layer**

Prepared by:

Zeyneb Benabdallah

Yassine Allouch

Supervisor:

Mme Hajer Taktak

Defended on 03/06/2026

School Year: 2025 / 2026

Acknowledgements

This work could not have been accomplished without the support and guidance of many people, to whom the authors wish to express their sincere gratitude.

The authors address their thanks first of all to their supervisor, **Mme Hajer Taktak**, for the trust she placed in them, for her guidance, her valuable advice and her availability throughout this project. Her insight greatly shaped both the technical direction and the quality of this work.

The authors also extend their gratitude to the members of the jury for the honour of agreeing to evaluate this project, as well as to the teaching staff of the *National Institute of Applied Science and Technology (INSAT)* for the knowledge shared throughout their studies.

Finally, the authors warmly thank their families and friends for their constant encouragement and unwavering support throughout this journey.

Abstract

This report presents the design and software architecture of **Abbes**, a child-sized, 3D-printed humanoid robot with eighteen degrees of freedom (18-DOF). The platform is built end to end: from the mechanical CAD model and the electronics organised around an ESP32 micro-ros node, to a complete ROS 2 software stack that handles kinematics, balance and bipedal-gait generation. A Gazebo simulation provides a digital twin used for validation before deployment on the physical robot. Beyond the classical motion-control pipeline, the project introduces a high-level *natural-language command layer*: a locally hosted Large Language Model grounds free-form user instructions into a fixed, safety-checked command vocabulary that drives the robot’s actions. Particular emphasis is placed on the software engineering aspects of the system — the embedded firmware, the ROS 2 control architecture and the artificial-intelligence command pipeline — while the mechatronic and gait-planning foundations of the platform are also covered.

Contents

Acknowledgements	i
Abstract	ii
1 General Presentation and State of the Art	1
1.1 Introduction	1
1.2 Problematic	1
1.3 State of the Art	2
1.3.1 WABIAN by Waseda	2
1.3.2 ASIMO by Honda	2
1.3.3 HRP Humanoid Series by AIST	3
1.3.4 Positioning of Abbes	4
1.4 Conclusion	4
2 Concepts on Bipedal Walking	6
2.1 Introduction	6
2.2 Walking Definition	6
2.3 Motion Planes	6
2.4 Robot Control and Stability	6
2.4.1 Centre of Mass (CoM)	7
2.4.2 Zero-Moment-Point (ZMP)	7
2.4.3 Support Polygon	7
2.4.4 Stability Criteria	8
2.5 Types of Bipedal Walking	8
2.6 Robot Gait Generation	9
2.6.1 Walking-Cycle Components	9
2.6.2 Walking-Cycle Phases	9
2.6.3 Gait Engine	9
2.7 Walking Primitive	10
2.8 Conclusion	10
3 Hardware and Mechatronic Design	11
3.1 Introduction	11
3.2 Mechanical Design	11
3.2.1 Anthropometric Model	11
3.2.2 Mechanical Articulated Structure	11
3.2.3 Final Design: Joint Limits and Resting Positions	12
3.3 Hardware Components	12
3.3.1 Actuators	12

3.3.2	Sensors	13
3.3.3	Power Supply	14
3.3.4	DC/DC Regulator	14
3.4	Electronic Architecture	15
3.5	Conclusion	15
4	Embedded Firmware: the On-Board Controller	17
4.1	Introduction	17
4.2	Choice of the ESP32	17
4.3	Firmware Architecture	18
4.4	Actuators and Sensors over I ² C	18
4.5	Joining ROS 2 with micro-ROS	19
4.6	Conclusion	21
5	Software Architecture and ROS 2 Integration	22
5.1	Introduction	22
5.2	What ROS 2 Is, and Why It Is Used Here	22
5.3	Workspace and Package Architecture	23
5.4	The Robot Model	23
5.5	Motion Control with <code>ros2_control</code>	24
5.6	Simulation with Gazebo	25
5.7	The Node and Topic Graph	26
5.8	Cross-Cutting Conventions	27
5.9	Conclusion	28
6	Kinematics and Gait Planning	29
6.1	Introduction	29
6.2	Leg Kinematics	29
6.3	Inverse Kinematics	29
6.4	Inertial Estimation	30
6.5	Balance Control	30
6.6	Gait Generation by ZMP Preview Control	31
6.7	The Complete Motion Pipeline	32
6.8	Results and Limitations	33
6.9	Conclusion	33
7	The Natural-Language Command Layer	34
7.1	Introduction	34
7.2	Design Principle: the LLM Never Moves a Motor	34
7.3	The Command Vocabulary and its Server	35
7.4	The Local Language Model and Tool-Use	35
7.5	The Dispatcher and the Feedback Loop	35
7.6	The Safety Validator	36
7.7	The Web Interface	36
7.8	End-to-End Behaviour	37
7.9	Conclusion	37

8	Simulation, Testing and Validation	39
8.1	Introduction	39
8.2	Simulation Environment and Method	39
8.3	Standing Stability	39
8.4	Balance Validation	39
8.5	Gait Validation	40
8.6	Command-Layer Validation	41
8.7	Results Summary	41
8.8	Conclusion	41
9	General Conclusion and Perspectives	43
9.1	Summary of the Work	43
9.2	Achievements and Limitations	43
9.3	Perspectives	44
9.4	Closing Remarks	44
	Bibliography	44

List of Figures

1.1	WABIAN by Waseda.	3
1.2	ASIMO.	4
1.3	HRP humanoid robots.	5
2.1	Motion planes: sagittal, horizontal, and frontal.	7
2.2	General case of the ZMP: (a) three-dimensional case at the ankle, (b) planar case where the moment M_{Ax} is balanced by shifting the point of action of the reaction force R_z	8
2.3	Support polygon in the single- and double-support phases.	8
3.1	Anthropometric model: segment lengths as a proportion of total height H (Winter’s model).	12
3.2	Robot morphology: joint arrangement and reference frame (P: pitch, R: roll, Y: yaw).	13
3.3	Mechanical design of Abbes: (a) CAD model and (b) the 3D-printed, assembled robot.	13
3.4	Electronic architecture of Abbes: the ESP32 micro-ROS node drives the 16 actuated servos through a single PCA9685 board and reads the MPU6050, communicating with the ROS 2 host over WiFi. Dashed red lines indicate power distribution.	15
4.1	Firmware data paths: the command path (top) drives the servos at 100 Hz, the sensing path (bottom) publishes the IMU at 50 Hz. Both periodic loops run on the microcontroller’s clock.	18
4.2	On-board electronics of Abbes: (a) the ESP32 with the PCA9685 servo driver and the servo wiring harness, and (b) the DC-DC regulators supplying the actuators.	19
4.3	micro-ROS bridges the microcontroller to the ROS 2 graph: a lightweight client on the ESP32 talks to an agent on the host, which connects it to the full DDS network.	20
4.4	Agent-connection state machine, providing automatic reconnection without rebooting the robot.	20
5.1	Internals of the <code>ros2_control</code> controller manager: the resource manager owns the command and state interfaces; each controller claims the interfaces for its joints, and the whole loop runs at 100 Hz.	24
5.2	Controller lifecycle (unloaded $\rightarrow$ configured $\rightarrow$ active) and the steps a spawner performs to bring a controller online.	25

5.3	Integration of Gazebo and <code>ros2_control</code> : the <code>gz_ros2_control</code> plugin connects the simulated joints to the same interfaces used on hardware, while <code>ros_gz_bridge</code> relays sensor topics into ROS 2.	26
5.4	How the <code>gz_ros2_control</code> plugin operates inside the Gazebo process: it binds the declared joints to simulation entities and exchanges state and commands on every step.	27
5.5	The complete node and topic graph: the planning layer feeds the controller manager, which drives either the Gazebo simulation path or the real-robot (ESP32) path; both share the downstream sensor and state topics.	28
6.1	Geometric inverse kinematics of the leg, decomposed into the sagittal plane (knee and hip/ankle pitch) and the frontal plane (hip and ankle roll). . . .	30
6.2	Walking in the sagittal (a) and frontal (b) planes: the centre-of-mass trajectory and the ZMP evolve across the double-support (DS) and single-support (SS) phases.	31
6.3	Swing-foot trajectory over the planned footstep sequence produced by the walking-pattern generator.	32
6.4	The complete motion pipeline: a feed-forward ZMP gait chain feeds the inverse kinematics; the balance controller and the capture-point adjustment close the loop with inertial feedback.	32
7.1	The natural-language command layer: the model chooses a command, the safety validator vets it, and only the deterministic command server actuates the robot. Robot status feeds back to inform the next decision.	34
7.2	The dispatcher’s bounded agentic loop: each tool result is fed back to the model, which can react and retry until it produces a final natural-language reply (subject to a turn limit).	36
7.3	The Abbes web control panel: a dependency-free browser interface that drives the natural-language command layer.	37
8.1	The robot standing in the Gazebo simulation, used as the baseline for the balance and gait tests.	40

List of Tables

1.1	Comparison of ASIMO Robot Models.	3
1.2	Comparison of HRP Robot Models.	4
1.3	Comparison of Bipedal Robots.	5
3.1	Robot joint structure: axes, angular limits and resting positions (servo degrees).	14
3.2	Actuator specifications.	14
3.3	Sensor description.	15
3.4	Battery specifications.	15
3.5	Regulator specifications.	16
8.1	Summary of validation results.	42

Chapter 1

General Presentation and State of the Art

1.1 Introduction

Humanoid robotics is one of the most demanding fields of modern engineering: it combines mechanical design, real-time control, embedded electronics, and, increasingly, artificial intelligence into a single integrated system. Reproducing a behaviour as ordinary as human walking requires a machine to balance on a small support area while continuously coordinating many actuators, all under strict timing constraints. This makes a humanoid platform an ideal testbed for studying the interaction between low-level real-time control and high-level decision making.

This chapter introduces the project by outlining its objectives and its significance in the field of bipedal robotics. It then discusses the problematic aspects associated with bipedal walking, presents the state of the art through a few representative humanoid robots, and finally positions the **Abbes** robot — the platform developed in this work — with respect to those references. The objectives pursued are:

1. Design and assemble an 18-DOF humanoid structure from 3D-printed parts driven by hobby servo-motors.
2. Implement the on-board real-time firmware that actuates the joints and streams inertial measurements.
3. Build a modular ROS 2 software stack covering the kinematic model, balance control, and bipedal-gait generation.
4. Validate the robot in a physics-based simulation (digital twin) before deployment on hardware.
5. Add a natural-language command layer, based on a locally hosted Large Language Model, that translates free-form user requests into a fixed and safety-checked set of robot actions.

1.2 Problematic

Bipedal walking poses a unique set of challenges that stem from the need to maintain dynamic stability on two legs. For the platform developed here, these challenges are

tightly coupled with the software-engineering goals of the project. The core problematic aspects are:

- **Mechanical design.** Defining a joint distribution that is rich enough to produce a stable, human-like gait while remaining buildable with low-cost, 3D-printed parts and hobby servo-motors.
- **Control system.** Despite the need for inverse kinematics in controlling a walking humanoid, ensuring robustness and adaptability in real-time control algorithms running on a resource-constrained microcontroller remains a significant challenge.
- **Stability criteria.** Keeping the robot balanced during the various phases of walking, which requires a recognised stability criterion and an inertial feedback loop.
- **ROS 2 integration.** Leveraging the advanced integration capabilities of ROS 2 to bridge a physics-based simulation and the physical robot through a common middleware.
- **High-level interaction.** Allowing a user to command the robot in natural language while guaranteeing that only safe, well-defined motions are ever executed.

1.3 State of the Art

Existing humanoid robots around the world showcase the remarkable advancements in robotics technology, demonstrating capabilities ranging from human-like motion and interaction to replacing humans in various industrial areas. The following sections review a few representative platforms.

1.3.1 WABIAN by Waseda

Since 1967, Waseda University has been at the forefront of biped-robot research. WABIAN-2 features 41 mechanical degrees of freedom, including two 6-DoF legs, a 2-DoF waist, a 2-DoF trunk, two 7-DoF arms, two 3-DoF hands, a 3-DoF neck, and two 1-DoF feet with passive toe joints. Standing at 1480 mm tall and weighing 64 kg, WABIAN-2 achieves a human-like gait characterised by stretched knees, heel-contact, and toe-off motion. This is made possible by its foot mechanism with passive toe joints and a 2-DoF (Roll, Yaw) waist that emulates human pelvic movement.

1.3.2 ASIMO by Honda

ASIMO is a research robot and is not available for commercial sale; it has, however, been loaned for unique events and used by major companies as a reception host. As of February 2009, there were over 100 ASIMO robots in various versions, ultimately aiming to assist individuals who are disabled, elderly, or ill, and to perform tasks that are dangerous for humans. The fifth version of ASIMO, released in September 2011, features enhanced capabilities including the use of all its fingers, equipped with tactile sensors in the palm and force detectors in each finger; combined with object-recognition technology, it can pick up a bottle and unscrew its cap, or hold a paper cup without crushing it. One of its main innovations is the ability to change direction smoothly while walking, without interrupting its stride.



Figure 1.1: WABIAN by Waseda.

Table 1.1: Comparison of ASIMO Robot Models.

Project	Date	Weight	Length	Anatomy	Functions	DoF	DoF Head
P-1	1993–1997	175 kg	1.915 m	Head, Arm, Hand, Leg, Feet, Trunk	Walking	–	–
P-2	–	210 kg	1.82 m	–	Walking, Climbing stairs	30	–
P-3	–	52 kg	1.2 m	–	Speed walking (1.6 km/h)	26	–
Asimo I	2000	54 kg	1.3 m	–	Speed walking (3 km/h), Object detection, Face recognition	34	3
Asimo II	2005	54 kg	1.3 m	–	Running (6 km/h), Gripping objects	34	3
Asimo III	2011	48 kg	1.3 m	–	Human daily tasks, Collision avoidance, Dynamic path planning	57	3

1.3.3 HRP Humanoid Series by AIST

The Humanoid Robotics Project (HRP) is an initiative aimed at developing general domestic-helper robots, funded by Japan’s Ministry of Economy, Trade and Industry (METI) and the New Energy and Industrial Technology Development Organization (NEDO). The project is led by Kawada Industries with support from the National Institute of Advanced Industrial Science and Technology (AIST) and Kawasaki Heavy Industries. The HRP series, also known as Promet, began with modifications to Honda’s P3 robots and includes features such as a tele-command system. Notably, HRP-2 can stand up after lying flat, a capability lacking in Honda’s ASIMO. The HRP-4, a bipedal humanoid, stands 150 cm tall and weighs 39 kg.



Figure 1.2: ASIMO.

Table 1.2: Comparison of HRP Robot Models.

Project	Date	Weight	Length	Anatomy	Functions	DoF	DoF Head	DoF Arm	DoF Leg
HRP-1	1998	120 kg	1.6 m	Head, Trunk, Arm, Hand, Leg, Feet	Walking / Hand operations	30	2	14: 3 shoulders, 1 elbow, 3 wrist	12
HRP-2	2003	58 kg	1.54 m	–	Walking on rug, Standing, Human interaction	30	2	12: 3 shoulders, 1 elbow, 2 wrist	12
HRP-3	2007	65 kg	1.6 m	–	Speed walking 2 km/h	42	2	12: 3 shoulders, 1 elbow, 2 wrist	12
HRP-4C	2009	43 kg	1.58 m	–	Anthropomorphic, Face expressions	42	11: 3 neck, 8 face	4	12
HRP-4	2010	39 kg	1.51 m	–	Athleticism, Running, Operating tasks	34	2	4	12

1.3.4 Positioning of Abbes

The platforms above achieve outstanding performance but rely on custom, expensive hardware that is out of reach for a student project. Abbes is positioned instead as a low-cost, fully 3D-printed, 18-DOF humanoid built around hobby servo-motors and an ESP32 microcontroller. Its distinguishing contribution is not raw locomotion performance but its *software*: a modern ROS 2 control stack with a physics-based digital twin, and a natural-language command layer driven by a locally hosted Large Language Model — a feature uncommon on platforms of this scale, and the main software-engineering contribution of this work.

1.4 Conclusion

This chapter explored the fundamental principles and obstacles within bipedal robotics. By examining both the challenges and the state-of-the-art developments, it identified the critical focal points for the project. The subsequent chapters delve into the specific methodologies, design considerations, and practical implementations required to build the robot. Table 1.3 provides a concise overview of the key features of the robots studied above.



(a) HRP-1



(b) HRP-2

Figure 1.3: HRP humanoid robots.

Table 1.3: Comparison of Bipedal Robots.

Robot	Height (mm)	Number of DoF	Capabilities and Features
WABIAN by Waseda	1480	41	Human-like gait, stretched knees, heel-contact, toe-off motion
ASIMO by Honda	Varies	Varies	Bipedal locomotion, object manipulation, human interaction
HRP Series by AIST	Varies	Varies	Anthropomorphic design, various locomotion modes

Chapter 2

Concepts on Bipedal Walking

2.1 Introduction

Bipedal walking, or walking on two legs, is a complex form of locomotion that involves the coordination of multiple joints to achieve stability, balance, and forward motion. Understanding its principles is essential for designing and controlling a humanoid robot. This chapter introduces the fundamental concepts of bipedal walking: the definition of walking, the motion planes, the centre of mass, the Zero-Moment-Point, the support polygon, the stability criteria, the types of walking, and the gait cycle that together underpin the locomotion work presented later in this report.

2.2 Walking Definition

Walking can be defined as the action of moving by successive movements and support of the legs and feet without ever leaving the ground entirely. This definition does not constrain the number of limbs, so walking may be bipedal, quadrupedal, and so on; in this work the term refers to *bipedal* walking, since the objective is to reproduce human locomotion. Walking is therefore a cyclic phenomenon: each foot alternately swings through the air and then serves as support, either alone or together with the other foot. This cyclic nature is formalised through the notion of gait *cycles* and *phases*, developed in the following sections.

2.3 Motion Planes

The motion planes describe the movements associated with walking, as shown in Figure 2.1. The **sagittal** plane corresponds to a side view of the walker, the **horizontal** plane to a top view, and the **frontal** plane to a front view. The walker moves primarily in the sagittal plane, but movements such as hip sway, lateral displacement, and pelvic rotation occur in the other two planes and enhance the fluidity of walking.

2.4 Robot Control and Stability

Bipedal walking relies on several key concepts that ensure stability and efficiency. These concepts are essential for understanding how to design and control a bipedal robot.

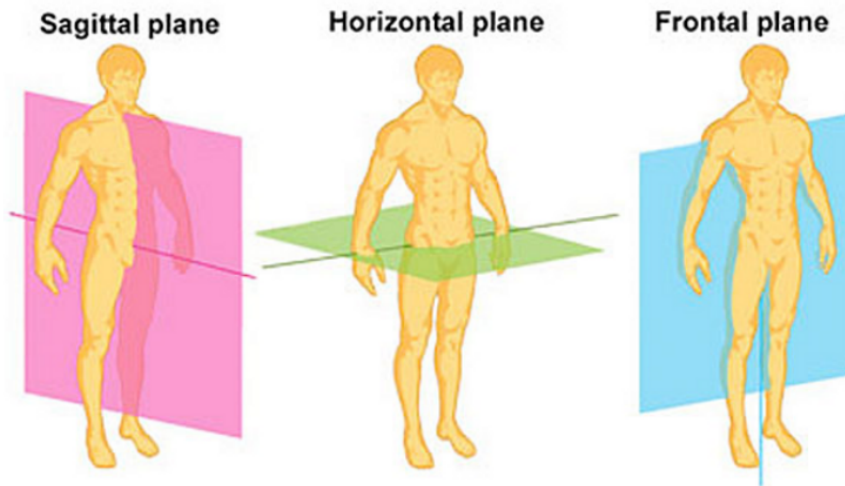


Figure 2.1: Motion planes: sagittal, horizontal, and frontal.

2.4.1 Centre of Mass (CoM)

The centre of mass (CoM), also known as the centre of gravity, is the point at which the mass of a body or system of bodies can be considered to be concentrated. For a rigid body, it is the point at which the body can be balanced without rotation under the influence of gravity alone. Mathematically, it is the weighted average of the positions of all the mass elements of the system. The CoM plays a central role in the study of the mechanics, dynamics, and stability of a walking robot.

2.4.2 Zero-Moment-Point (ZMP)

The Zero-Moment-Point (ZMP) is a criterion used to analyse the stability of a bipedal or multi-legged robot during dynamic motion. The ZMP is the point on the ground where the total moment of the forces acting on the robot's feet has no horizontal component — that is, the point where the ground-reaction forces and the inertial forces acting on the robot's body balance each other out.

Consider the support foot during the single-support phase (Figure 2.2). The effect of the parts above the support foot can be replaced by a force $\mathbf{F}$ and a moment $\mathbf{M}_A$ at the ankle, while the ground reaction $\mathbf{R}$ and moment $\mathbf{M}_S$ balance the system at the contact point S . Because the ground can only act on the foot through friction and a normal force, the horizontal moments vanish at this point — hence the term *Zero-Moment-Point*. The ZMP is thus the point under the foot where the ground reaction balances the horizontal moments applied at the ankle. To maintain dynamic balance, the ZMP must remain inside the support polygon; if the support area is too small to compensate the horizontal moment, the robot loses dynamic balance and the ZMP is no longer defined (a fictitious ZMP may then be computed for analysis).

2.4.3 Support Polygon

The support polygon is one of the stability criteria for humanoid walking. It is defined by the outer contour of the points of contact between the robot's feet and the ground. As

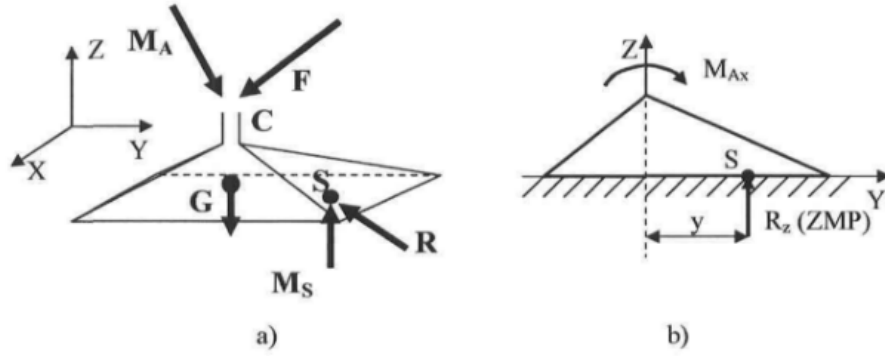


Figure 2.2: General case of the ZMP: (a) three-dimensional case at the ankle, (b) planar case where the moment M_{Ax} is balanced by shifting the point of action of the reaction force R_z .

illustrated in Figure 2.3, it takes two forms during the walking cycle: the **single-support** polygon, reduced to the contour of a single foot, and the larger **double-support** polygon, spanning both feet when they are simultaneously in contact with the ground.

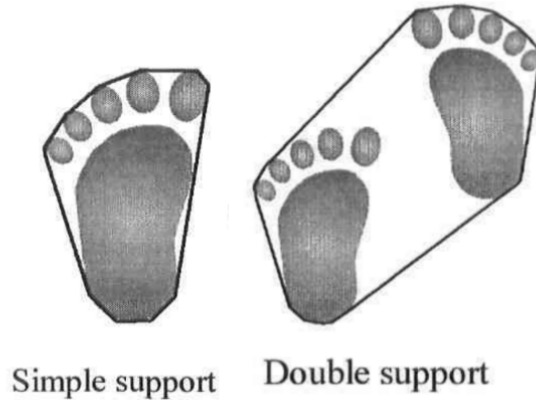


Figure 2.3: Support polygon in the single- and double-support phases.

2.4.4 Stability Criteria

Two complementary aspects of stability are considered:

- **Posture stability** — the ability to maintain a stable standing position with minimal jerk. Joint positions and the centre of gravity are the main variables involved.
- **Gait stability** — the ability to maintain balance during locomotion. The key variables here are step length, foothold positions, and ZMP data.

2.5 Types of Bipedal Walking

Walking can be categorised into two types:

- **Static walking.** The posture is stable at all times: the body weight is always supported by at least one foot, and the projection of the CoM on the ground always remains within the support polygon.
- **Dynamic walking.** A succession of stable and unstable postures. Instability is deliberately created forward when the projection of the CoM moves outside the support polygon. Dynamic walking is the natural human gait; the resulting instability is corrected by the next step.

2.6 Robot Gait Generation

2.6.1 Walking-Cycle Components

Ignoring the very first and very last steps to and from a standing posture, the walking cycle is composed of two alternating phases:

Double-Support Phase (DSP) The robot is in contact with the floor through both legs. This phase lets the robot shift its ZMP into a favourable position for the next step without losing balance; it is therefore called the *stabilisation* phase.

Single-Support Phase (SSP) Also known as the swing phase, the robot is in contact with the floor through a single leg. The leg in contact is the *support* leg and the other is the *swinging* leg.

2.6.2 Walking-Cycle Phases

Within these components, the cycle progresses through the following phases:

- **Impact / contact phase (DSP)** — the heel of the front foot comes into contact with the floor.
- **Lift-off / swing phase (SSP)** — during the transition from double to single support, the swinging foot is lifted off the ground; the robot transfers its weight onto the support foot and starts swinging the rear leg forward.
- **Catch phase (DSP)** — the swinging foot comes back into contact with the floor, returning the robot to double support.

The roles of the two legs are then swapped and the cycle repeats.

2.6.3 Gait Engine

The gait engine, also referred to as the walking-pattern generator, is the software component responsible for producing a sequence of feasible joint positions that reproduce a walking motion. It accounts for the robot's physical limitations, its preferred walking speed, and the desired gait style to compute the best sequence of joint angles and body motions for stable and efficient walking. Gait engines fall into two main families:

- **ZMP-based approach** — requires comprehensive knowledge of the robot's parameters and uses the ZMP as a feedback signal to adjust joint angles and torques.
- **Inverted-pendulum approach** [1] — operates with limited knowledge of the robot model and uses a closed-loop feedback based on the inverted-pendulum abstraction.

The gait generator developed in this work follows the ZMP-based approach and is detailed in Chapter 6.

2.7 Walking Primitive

A walking primitive [2] is a fundamental building block that generates the motion of a robot’s gait. It defines characteristics such as step length, step height, and step frequency, taking into account the robot’s physical constraints, its stability requirements, and the desired locomotion behaviour.

2.8 Conclusion

This chapter introduced the fundamental concepts of bipedal walking, providing the vocabulary and the stability criteria — centre of mass, Zero-Moment-Point, and support polygon — that govern stable, human-like locomotion, together with the structure of the gait cycle. These concepts form the theoretical foundation for the hardware, firmware, and control chapters that follow. The next chapter presents the hardware and mechatronic design of the Abbes robot.

Chapter 3

Hardware and Mechatronic Design

3.1 Introduction

Designing a bipedal robot is a multidisciplinary task that combines principles from mechanical engineering, electronics, and computer science. This chapter presents the physical foundation of the **Abbes** robot: the mechanical design derived from human proportions, the selection of actuators and sensors, the power supply, and the electronic architecture built around an ESP32 microcontroller. Together these subsystems form the platform on which the firmware and control software of the following chapters run.

3.2 Mechanical Design

The mechanical design is inspired by the anatomical structure of the human body. As the review of existing humanoids in Chapter 1 showed, the construction of humanoid robots is rooted in mimicking human morphology.

3.2.1 Anthropometric Model

The anthropometric model expresses the robot’s dimensions as a function of human-body proportions, so that its segments and movements stay as close to human anatomy as possible. Anthropometry — from the Greek *anthropos* (human) and *metron* (measure) — is the study and measurement of the physical dimensions of the human body. This work relies on *static* anthropometric measurements, whose implementation is simpler, using the normalised segmental proportions established by Winter. Figure 3.1 gives the length of each limb relative to the total height H of the individual.

3.2.2 Mechanical Articulated Structure

The articulated structure is the set of interconnected joints and links that lets the robot reproduce human-like movements. The number of degrees of freedom (DOF) must be large enough for agility and balance, yet kept minimal to simplify the model. For a leg, two DOF (at the hip and ankle) enable pendulum-like motion in the sagittal plane, and two more provide lateral motion in the frontal plane; a fifth DOF at the knee is required to clear the foot during the swing phase. This leads to five DOF per leg [4]. Each arm uses three DOF, and the head two, as detailed in the next section. Figure 3.2 shows the resulting joint arrangement.

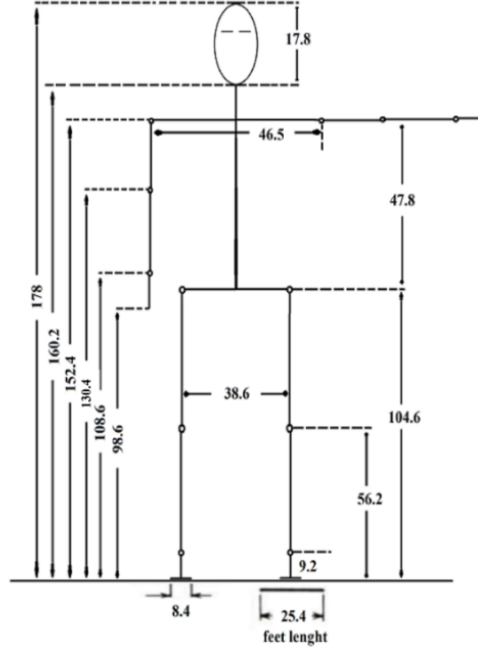


Figure 3.1: Anthropometric model: segment lengths as a proportion of total height H (Winter’s model).

3.2.3 Final Design: Joint Limits and Resting Positions

The final design is an 18-DOF humanoid with the following distribution:

- **Each leg — 5 DOF:** 2 at the hip (pitch and roll), 1 at the knee (pitch), and 2 at the ankle (pitch and roll).
- **Each arm — 3 DOF:** 2 at the shoulder (pitch and roll) and 1 at the elbow (roll).
- **Head — 2 DOF:** head yaw and camera pitch.

Table 3.1 summarises the complete joint structure. The angle range and resting position of each joint are expressed in degrees of servo travel; these exact values are the single source of truth used by the firmware (the limits are enforced before any command reaches a servo, see Chapter 4). The joints are listed in the firmware’s index order, which groups the right-side joints and the head on the first PWM board and the left-side joints on the second.

The mechanical parts are designed in CAD (Figure 3.3a) and manufactured by 3D printing; the assembled robot is shown in Figure 3.3b.

3.3 Hardware Components

3.3.1 Actuators

Selecting actuators involves several criteria: high torque and precision for accurate motion, efficient current consumption, a size and weight compatible with the structure, durability, and cost. Table 3.2 compares the candidates considered. The **MG996R** servo was selected: it offers a sufficient torque-to-cost ratio for a child-sized, 3D-printed platform,

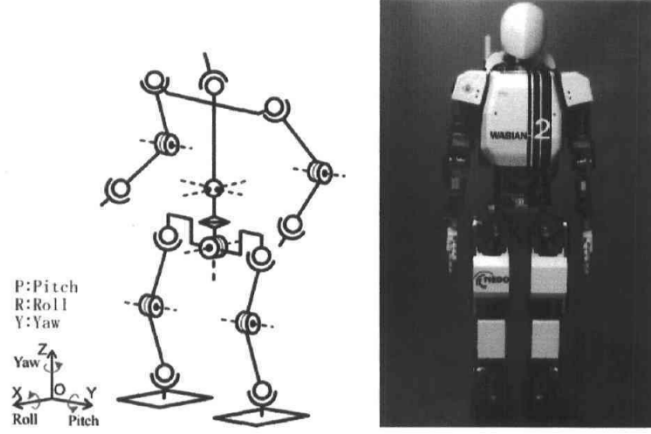


Figure 3.2: Robot morphology: joint arrangement and reference frame (P: pitch, R: roll, Y: yaw).

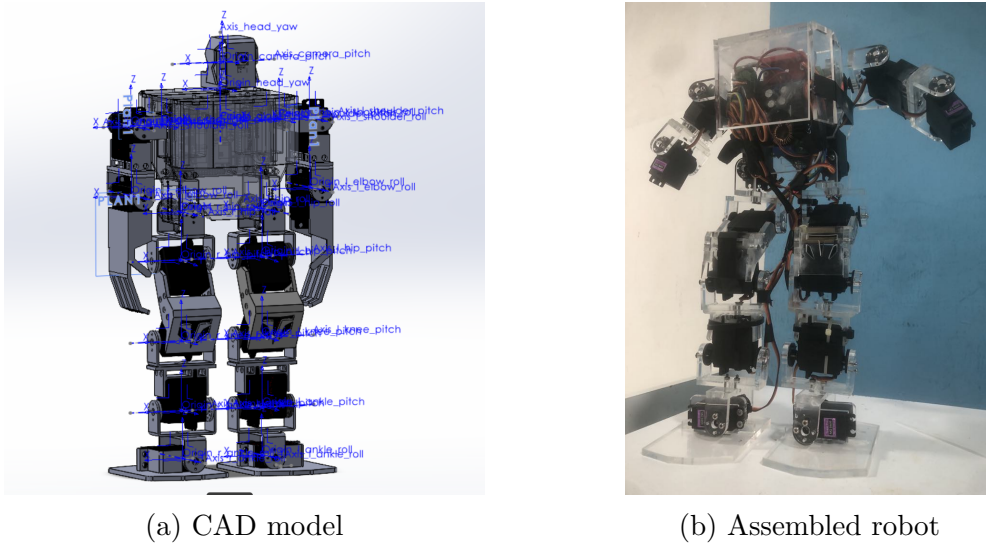


Figure 3.3: Mechanical design of Abbes: (a) CAD model and (b) the 3D-printed, assembled robot.

and it is directly compatible with standard PWM control. The robot uses eighteen such servos, one per joint.

3.3.2 Sensors

The robot's primary sensor is the **MPU6050**, which combines a three-axis gyroscope and a three-axis accelerometer in a single low-cost, I²C-compatible package (Table 3.3). It provides the angular-velocity and acceleration data needed to estimate the robot's orientation and to maintain balance during motion. Its digital interface simplifies integration with the microcontroller.

Table 3.1: Robot joint structure: axes, angular limits and resting positions (servo degrees).

#	Joint name	Axis	Min (°)	Max (°)	Rest (°)
1	r_shoulder_pitch	pitch	0	180	90
2	r_shoulder_roll	roll	0	180	90
3	r_elbow_roll	roll	0	180	90
4	head_yaw	yaw	0	180	90
5	camera_pitch	pitch	0	180	90
6	r_hip_roll	roll	0	170	50
7	r_hip_pitch	pitch	0	170	80
8	r_knee_pitch	pitch	0	170	100
9	r_ankle_pitch	pitch	10	180	100
10	r_ankle_roll	roll	20	150	90
11	l_shoulder_pitch	pitch	0	180	90
12	l_shoulder_roll	roll	0	180	90
13	l_elbow_roll	roll	0	180	90
14	l_hip_roll	roll	0	170	50
15	l_hip_pitch	pitch	0	170	80
16	l_knee_pitch	pitch	0	170	70
17	l_ankle_pitch	pitch	10	180	90
18	l_ankle_roll	roll	20	150	90

Table 3.2: Actuator specifications.

Actuator	Model	Price (€)	Torque (kg·cm)	Position sensor	Current (A)
Servo	MG996R	20	10	–	1
Dynamixel	MX-28	312	31.6	Contactless	1.7
Servo	HS-645MG	38.40	107 / 133	–	2.5

3.3.3 Power Supply

The power supply must deliver the voltage and current required by all components within budget constraints. The robot is *currently powered from an off-board source*; a battery-powered version is planned as future work. Table 3.4 lists the battery options selected for that version: a high-capacity LiPo pack is preferred for its current rating, at a higher cost than a basic lithium cell.

3.3.4 DC/DC Regulator

A DC/DC converter adapts the battery voltage to the levels required by the servos and the electronics. Table 3.5 compares the regulators considered, trading current rating against cost.

Table 3.3: Sensor description.

Sensor	Description
MPU6050	Combines a three-axis gyroscope and a three-axis accelerometer (raw I ² C).

Table 3.4: Battery specifications.

Battery	Current rating (mAh)	Cost
LiPo	5000	High
Lithium	1000	Low

3.4 Electronic Architecture

The robot’s electronics are organised into a **control section** and a **power section**.

The control section is built around an **ESP32 microcontroller**, which runs the real-time control loop directly on the robot and joins the ROS 2 graph as a *micro-ROS* node over WiFi/UDP. The ESP32 drives the servos through a **PCA9685** 16-channel PWM driver board on the I²C bus (at address 0x40). In the current build, sixteen of the eighteen joints are actuated — the ten leg joints and the six arm joints — which exactly fills the sixteen channels of one board; the two head joints (`head_yaw` and `camera_pitch`) are part of the kinematic model but are not actuated in this version. The same I²C bus connects the **MPU6050** IMU. On the host (laptop) side, a micro-ROS *agent* bridges the ESP32 into ROS 2, so that the inertial data published by the node and the joint commands it receives are available as ordinary ROS 2 topics. This architecture replaces the more common pairing of a single-board computer with a separate servo controller by a single, low-cost microcontroller that communicates wirelessly. The power section supplies the servos and electronics through the battery and DC/DC regulator described above. Figure 3.4 summarises the architecture.

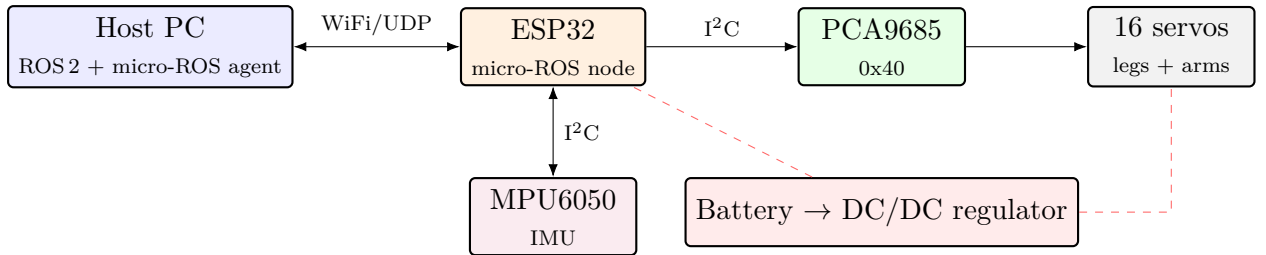


Figure 3.4: Electronic architecture of Abbes: the ESP32 micro-ROS node drives the 16 actuated servos through a single PCA9685 board and reads the MPU6050, communicating with the ROS 2 host over WiFi. Dashed red lines indicate power distribution.

3.5 Conclusion

This chapter presented the hardware foundation of Abbes. The mechanical structure was derived from anthropometric data to obtain a human-like, 18-DOF morphology, and its

Table 3.5: Regulator specifications.

Regulator	Current rating (A)	Cost (€)
XL6009	3.0	14
XL4016	9.0	30
LM2596	1.5	8

joint limits were defined (Table 3.1). The components were selected to balance performance, cost, and efficiency: MG996R servos, an MPU6050 IMU, and a battery/regulator pair for the powered version. The electronic architecture is built around an ESP32 that drives the actuated servos through a PCA9685 board and joins ROS 2 as a micro-ROS node. The next chapter details the embedded firmware that runs on this ESP32.

Chapter 4

Embedded Firmware: the On-Board Controller

4.1 Introduction

The firmware is the lowest software layer of the robot: it runs directly on the microcontroller introduced in Chapter 3 and turns the electronics into an active component of the robot’s software system. Its responsibilities are concrete: receive the eighteen target joint angles, drive the actuated servos smoothly through the PCA9685 board, read the MPU6050 inertial sensor, and make all of this available to the rest of the system. This chapter first justifies the choice of the ESP32, then explains how the firmware is organised, how it handles the actuators and sensors over the I²C bus, and finally how it joins the ROS 2 world through *micro-ROS* — the bridge that leads naturally into the next chapter.

4.2 Choice of the ESP32

The on-board controller has a demanding but bounded role: it must update eighteen servos and sample an inertial sensor at a steady rate, while exchanging data with the host computer. Selecting a microcontroller for this task involves balancing computing power, connectivity, and cost.

The **ESP32** was chosen because it meets all of these constraints at once. It is a low-cost, dual-core processor running at 240 MHz with enough memory to host a lightweight ROS 2 client, which is far beyond the reach of an 8-bit board such as a basic Arduino. Crucially, it integrates **WiFi**, allowing the robot to communicate with the host wirelessly and to operate untethered. It natively supports the **I²C** bus required by both the PCA9685 PWM drivers and the MPU6050 sensor, and it runs a real-time operating system (FreeRTOS) whose two cores can separate time-critical communication from supervision. Finally, it is well supported by the embedded ROS 2 tooling used in this project. Compared with a single-board computer such as a Raspberry Pi, the ESP32 is cheaper, lighter, and consumes far less power — and the modest computation performed on board does not require a full operating system, making the single-board computer unnecessary for this layer.

4.3 Firmware Architecture

The firmware is written in C++ on the Arduino/PlatformIO framework and is deliberately split into focused modules rather than a single monolithic program: one module manages the servos, one is a self-contained driver for the IMU, and one handles the communication with the host. A separate file holds the joint map (Table 3.1) and all hardware constants, so that no hardware-specific value is hidden in the logic.

The architecture takes advantage of the ESP32’s two cores. All communication — the wireless link, the connection lifecycle, and the dispatching of incoming and outgoing messages — runs in a dedicated real-time task pinned to **core 1**. **Core 0** is left to the main program, which only supervises the system and is reserved for future watchdog and safety logic. Separating communication from supervision in this way ensures that activity on the network cannot stall the safety logic, and vice versa.

Functionally, the firmware implements two continuous data paths, shown in Figure 4.1. On the *command path*, target joint angles arriving from the host are stored, and a periodic 100 Hz routine drives the servos toward those targets through the PWM boards. On the *sensing path*, the inertial sensor is sampled and a periodic 50 Hz routine sends the measurements back to the host. Both periodic routines are driven by the microcontroller’s own clock, so the joint-control rate remains regular even when network traffic is not. On start-up, the same servo mechanism first brings all joints smoothly to their resting position before any command is accepted.

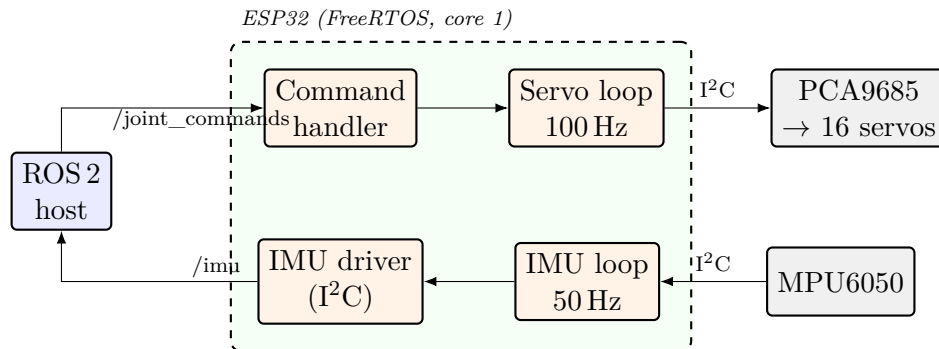
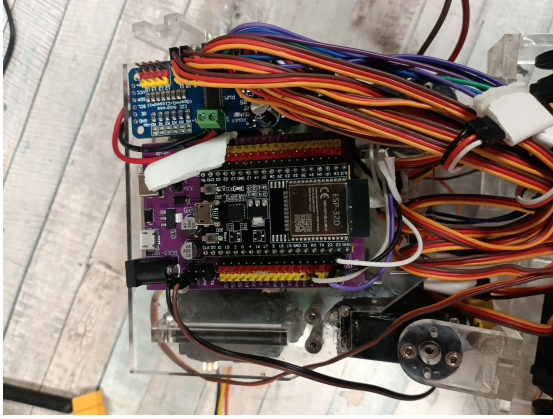


Figure 4.1: Firmware data paths: the command path (top) drives the servos at 100 Hz, the sensing path (bottom) publishes the IMU at 50 Hz. Both periodic loops run on the microcontroller’s clock.

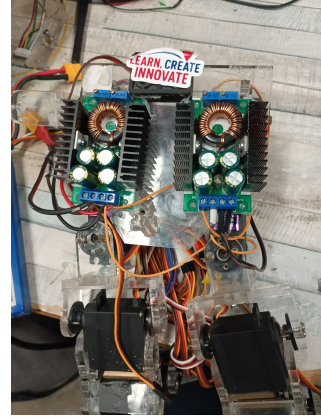
The on-board electronics that implement this architecture are shown in Figure 4.2: the ESP32 controller with its servo-driver board and wiring harness, and the DC-DC regulators that power the actuators.

4.4 Actuators and Sensors over I²C

The PWM driver boards and the inertial sensor all share a single **I²C** bus, a two-wire serial bus on which each device has its own address. This is what allows the peripherals — the PCA9685 PWM driver (at address 0x40) and the MPU6050 — to be controlled through only two microcontroller pins.



(a) ESP32 controller and PWM servo driver



(b) DC-DC power regulators

Figure 4.2: On-board electronics of Abbes: (a) the ESP32 with the PCA9685 servo driver and the servo wiring harness, and (b) the DC-DC regulators supplying the actuators.

Actuators. A joint angle cannot be sent to a servo directly; it must be expressed as the width of a PWM pulse. The firmware converts each angle in degrees to the corresponding pulse width for the MG996R servos and sends it to the appropriate PCA9685 channel. Of the eighteen joints in the command vector, sixteen are wired to the board’s channels (the ten leg joints and the six arm joints); the two head joints are still accepted in the command but have no servo in the current build and are simply skipped. Two safeguards are applied in the process. First, every command is *clamped* to the individual angular limits of the joint (Table 3.1) before it reaches a servo, so an out-of-range request can never force a joint past its mechanical limit. Second, and most importantly, the servos are never commanded to jump straight to a new target: the target is only stored, and the 100 Hz loop moves each joint toward it by a small, fixed maximum step on every tick — a *velocity-limited interpolation*. This ramps every motion smoothly and protects the plastic gears of the hobby servos from the abrupt slams that a sudden set-point would otherwise cause.

Sensor. The MPU6050 combines a three-axis accelerometer and a three-axis gyroscope, and is read over the same I²C bus through a self-contained driver rather than a third-party library, keeping the behaviour fully under control. At start-up the driver wakes the sensor, enables a low-pass filter to reject vibration noise, and sets the measurement ranges and sampling rate. Each reading then retrieves all of the acceleration and angular-velocity data in a single bus transaction, which guarantees that every axis belongs to the same sample, and converts the raw values into physical units. Importantly, the firmware performs *no orientation estimation on the device*: it transmits acceleration and angular velocity only. Fusing those two signals into a usable tilt estimate is left to a node on the host side (Chapter 5), which keeps the microcontroller’s workload light and places the filtering logic where it is easiest to tune.

4.5 Joining ROS 2 with micro-ROS

The two data paths described above must ultimately connect to the rest of the system. The entire software stack of Abbes is built on ROS 2 — the simulation, the controllers,

and the artificial-intelligence layer of the following chapters all communicate through it. ROS 2, however, is designed to run on full computers: its communication relies on the DDS middleware, whose memory and processing requirements far exceed what a microcontroller can provide. Keeping the ESP32 outside ROS 2 and talking to it over a custom serial protocol would mean hand-written message framing on both sides, no introspection, and brittle, hard-to-extend code.

micro-ROS resolves this. It brings the ROS 2 client concepts — nodes, topics, publishers and subscribers — to microcontrollers running a real-time operating system. In place of the full DDS middleware it uses a lightweight client that communicates with a companion process on the host, the **micro-ROS agent**; the agent translates the compact messages from the microcontroller into the full ROS 2 traffic, and back (Figure 4.3). From the point of view of the rest of the system, the ESP32 then appears as an ordinary ROS 2 node: its data can be inspected with the same tools and uses the same standard message types as any other node.

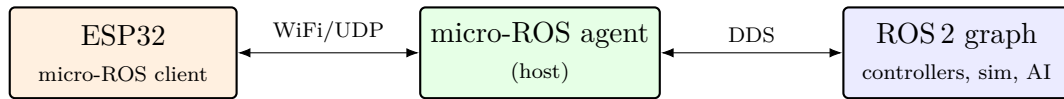


Figure 4.3: micro-ROS bridges the microcontroller to the ROS 2 graph: a lightweight client on the ESP32 talks to an agent on the host, which connects it to the full DDS network.

This is exactly why micro-ROS is convenient here: the on-robot controller speaks the same language as everything else, so the inertial data and the joint commands are exposed as standard ROS 2 topics, with no custom protocol or translation glue to maintain. Concretely, the firmware advertises a node that *subscribes* to the joint-command topic and *publishes* the IMU topic. Both use **best-effort** delivery: for a continuous stream of set-points and inertial samples, the newest value always supersedes the previous one, so re-transmitting a lost message would only add latency.

Because the wireless link can drop, the firmware tolerates an agent that comes and goes through the explicit state machine of Figure 4.4. It waits for the network, then for the agent; once the agent answers it creates the ROS 2 entities and enters the connected state, where it periodically re-checks the link. If the agent is lost, the node cleanly destroys its entities and returns to waiting — so the robot reconnects automatically, without a reboot. An on-board LED mirrors this state, lit only while connected.

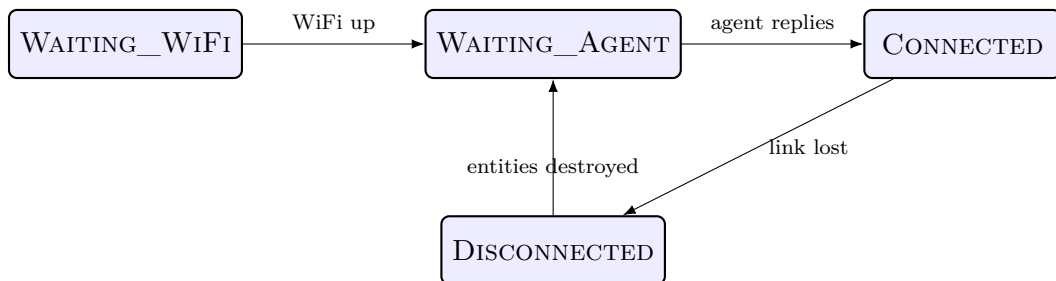


Figure 4.4: Agent-connection state machine, providing automatic reconnection without rebooting the robot.

4.6 Conclusion

This chapter described the embedded firmware of Abbas. The ESP32 was chosen for its dual-core processing, integrated WiFi, native I²C support, and low cost. Around it, the firmware is organised into focused modules running two periodic data paths — a 100 Hz servo loop and a 50 Hz sensing loop — with per-joint limiting and velocity-limited interpolation to protect the hardware. The actuators and the inertial sensor share a single I²C bus, and the node joins the rest of the system through micro-ROS, which makes the microcontroller a first-class participant in the ROS 2 graph. That graph — the host-side software architecture that produces the joint commands and consumes the inertial data — is the subject of the next chapter.

Chapter 5

Software Architecture and ROS 2 Integration

5.1 Introduction

The previous chapters described the physical robot and the firmware that animates it. This chapter presents the software that orchestrates the whole system on the host computer. Because this project is carried out within a software-engineering curriculum, this is the central chapter of the report: it explains what ROS 2 is and why it was adopted, how the code is organised into packages, how the robot is modelled and controlled through `ros2_control`, how it is simulated in Gazebo, and how all the pieces connect into a single, coherent system that drives both the simulation and the real robot.

5.2 What ROS 2 Is, and Why It Is Used Here

Despite its name, ROS 2 (Robot Operating System 2) is not an operating system but a *middleware* and a set of conventions for building robot software as a distributed system. Rather than one large program, an application is structured as a graph of small, independent processes called **nodes**, each responsible for a single concern. Nodes never call each other directly; they communicate through named channels using three complementary patterns:

- **Topics** — anonymous publish/subscribe streams, used for continuous data such as the inertial measurements or the joint states;
- **Services** — synchronous request/response calls, used for short queries such as asking the robot for its status;
- **Actions** — long-running, cancellable goals that provide progress feedback, used for goal-driven motions such as walking or bowing.

Underneath, ROS 2 relies on the DDS standard for decentralised discovery and for configurable quality-of-service, so nodes find one another automatically and can tune reliability per channel. The source code is organised into **packages** gathered in a **workspace** and built with the `colcon` tool.

This model is precisely what makes ROS 2 the right foundation for Abbes. First, it imposes a clean *modular* architecture: inverse kinematics, balance, gait generation, inertial filtering and the high-level command interface each live in their own node and

can be developed, tested and replaced in isolation. Second, the standardised message types and the rich tooling (live topic inspection, graph visualisation, recording and replay, the RViz 3-D viewer) make the system observable, which is invaluable when debugging a robot. Third, and most importantly for this project, the *same* interfaces drive both the Gazebo simulation and the physical robot: an algorithm validated against the digital twin can be deployed on hardware without rewriting it. Fourth, `ros2_control` provides a standard hardware-abstraction layer (Section 5.5), and micro-ROS (Chapter 4) extends the very same graph down to the microcontroller. A monolithic control program would offer none of this: it would be hard to test in parts, opaque at runtime, and impossible to retarget from simulation to hardware without major rewrites.

5.3 Workspace and Package Architecture

The software is organised as a ROS 2 workspace whose responsibilities are split across five packages, each with a clear role:

- `my_robot_description` — the *model* layer. A purely static package describing the robot’s physical reality (kinematic chain, joint axes and limits, link masses, collision geometry, visual meshes) in URDF/Xacro. Every other package depends on it.
- `my_robot_bringup` — the *launch* layer. It contains no algorithm; its role is orchestration: load the model, start the Gazebo simulation, configure the controllers, and bridge the simulator to ROS 2.
- `humanoid_robot` — the *algorithm* layer. The nodes that actually compute: inertial filtering, balance control, inverse kinematics, and the walking-pattern generator (Chapter 6).
- `humanoid_msgs` — the *interface* layer. The action and service definitions (`Walk`, `StandStill`, `SitDown`, `WaveHand`, `Bow`, `Stop`, `GetStatus`) shared across the system, so that interfaces are never re-defined ad hoc.
- `humanoid_command_api` — the *high-level* layer that exposes the robot’s command vocabulary and hosts the natural-language interface (Chapter 7).

This separation mirrors the standard ROS 2 practice of decoupling the description, the bring-up, the algorithms and the interfaces, so that a change in one (for example, a new gait algorithm) does not ripple through the others.

5.4 The Robot Model

The `my_robot_description` package is the single source of truth for the robot’s geometry. The model is written in Xacro — a macro language that expands to URDF — and was exported from the Onshape CAD assembly, so that the simulated kinematics match the manufactured parts. It declares the kinematic chain of revolute joints (the eighteen actuated degrees of freedom plus fixed joints for the structural links), each with its axis, parent and child links, angular limits, and the inertial and collision properties needed for a faithful physics simulation. A companion file defines the visual materials, and an RViz configuration provides a consistent debugging view. Because every other package depends

on this model, a single joint-name change here must propagate everywhere — a constraint revisited in Section 5.8.

Crucially, the model also embeds a `ros2_control` block declaring, for each joint, a position *command interface* and position and velocity *state interfaces*. This block is what connects the abstract robot description to the control framework described next.

5.5 Motion Control with `ros2_control`

Joint motion is handled by `ros2_control`, the standard ROS 2 control framework, rather than by ad-hoc code. Its central component is the **controller manager**, a single node that hosts all the controllers and owns a **resource manager**. The resource manager holds, for every joint, a *command interface* (a value to be written, here a target position) and *state interfaces* (values that can be read, here position and velocity). Controllers do not touch the hardware directly; they *claim* the interfaces they need from the resource manager, which guarantees that no two controllers fight over the same joint (Figure 5.1).

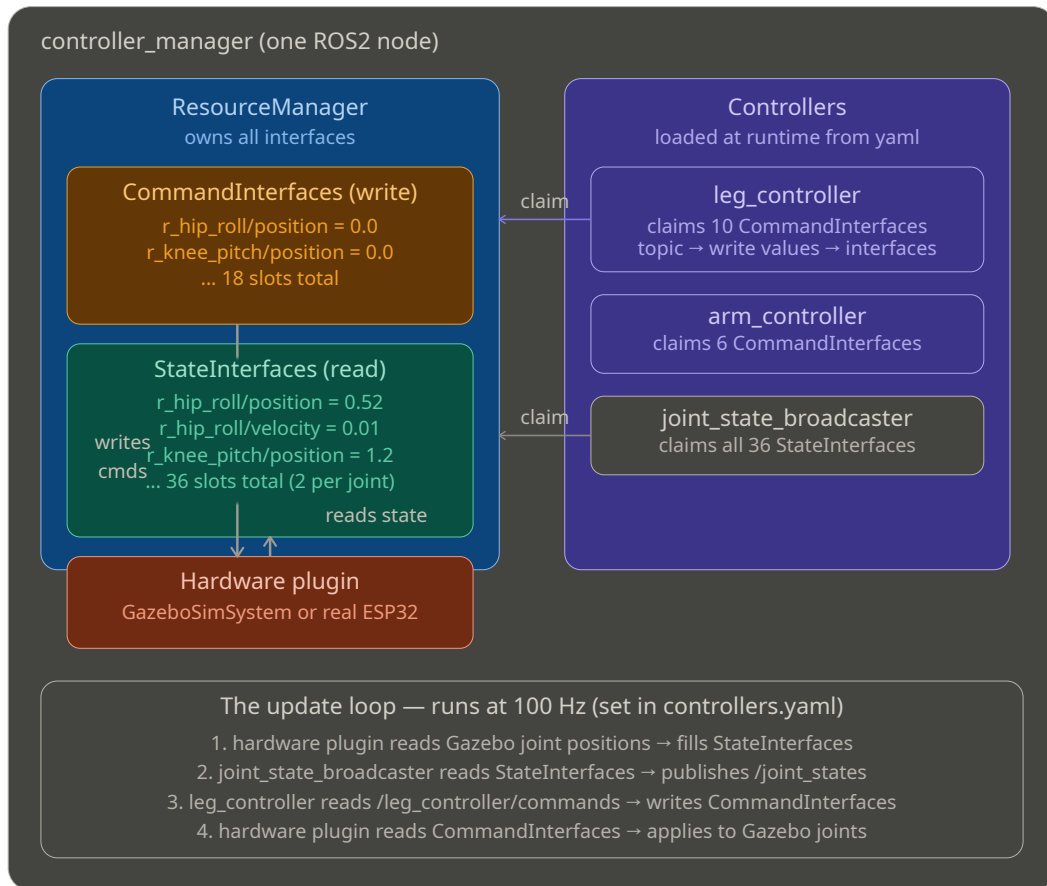


Figure 5.1: Internals of the `ros2_control` controller manager: the resource manager owns the command and state interfaces; each controller claims the interfaces for its joints, and the whole loop runs at 100 Hz.

For Abbes, the joints are grouped into three controllers, configured in `controllers.yaml` and all of the *forward command* type (they forward a commanded position to the position interface):

- `leg_controller` — the ten leg joints;
- `arm_controller` — the six arm joints;
- `head_controller` — the two head joints.

A fourth controller, the `joint_state_broadcaster`, is read-only: it reads every joint's state interfaces and publishes them on `/joint_states`. The whole loop runs at 100 Hz: on each cycle the hardware interface refreshes the state interfaces, the broadcaster publishes the joint states, each controller reads its command topic and writes the command interfaces, and the hardware interface applies them.

Controllers are not all active at start-up; they follow a lifecycle (Figure 5.2). A *spawner* loads each controller from its configuration, asks the resource manager to grant the interfaces it requires, and then activates it, at which point the controller begins listening on its command topic and participates in the update loop.

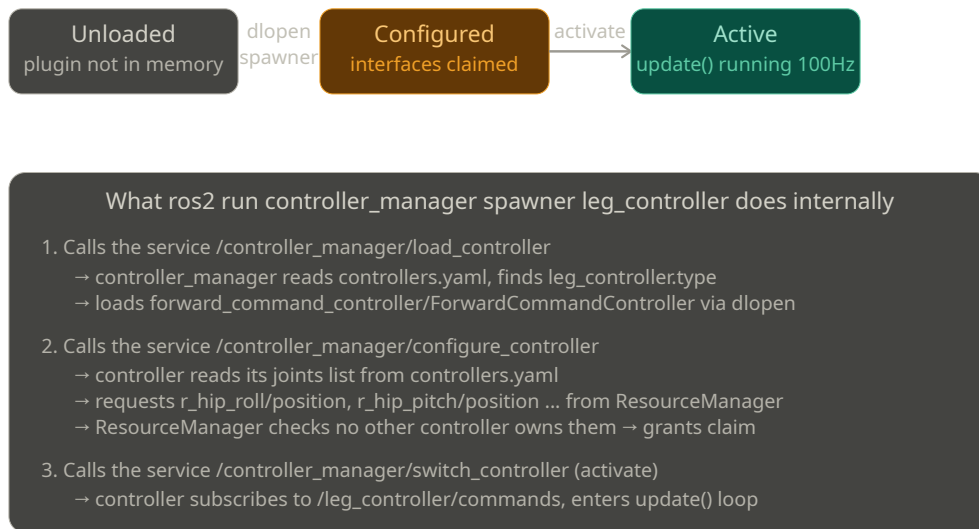


Figure 5.2: Controller lifecycle (unloaded → configured → active) and the steps a spawner performs to bring a controller online.

5.6 Simulation with Gazebo

Before any motion is attempted on the physical robot, it is validated on a *digital twin* in Gazebo Harmonic, whose DART physics engine simulates gravity, contacts and joint dynamics. The link between Gazebo and `ros2_control` is made by the `gz_ros2_control` plugin: the URDF declares the plugin and a `GazeboSimSystem` hardware interface, so that the simulated joints expose exactly the same command and state interfaces as a real robot would (Figure 5.3). As a result, the controllers described above are completely unaware of whether they are driving the simulator or hardware — which is the foundation of the sim-to-real strategy.

Internally, Gazebo represents the world as an entity-component system, and the `gz_ros2_control` plugin runs as part of the simulation loop: at start-up it parses the



Figure 5.3: Integration of Gazebo and `ros2_control`: the `gz_ros2_control` plugin connects the simulated joints to the same interfaces used on hardware, while `ros_gz_bridge` relays sensor topics into ROS 2.

`ros2_control` block of the URDF and binds each declared joint to its simulated entity; on every simulation step it reads the joint states into the state interfaces, lets the controllers run, and writes the resulting commands back to the simulated joints (Figure 5.4). The inertial sensor defined on the torso is exported to ROS 2 by the `ros_gz_bridge` as the `/imu` topic, mirroring the topic the firmware publishes on the real robot.

The `my_robot_bringup` package ties this together in a single launch sequence: it publishes the robot description (expanding the Xacro), starts Gazebo with an empty world, spawns the robot from its description, starts the `ros_gz_bridge` for the clock and IMU, and finally spawns the four controllers in order.

5.7 The Node and Topic Graph

Putting the pieces together yields the system graph of Figure 5.5. At the top, the algorithm nodes of the planning layer (inverse kinematics, balance, and the gait generator of Chapter 6) publish target joint positions, in radians, onto the three controller command topics. The controller manager applies them through the controllers described above. From there, the graph splits into two paths that share the same upstream:

- the **simulation path**, where the `gz_ros2_control` plugin drives the joints inside Gazebo’s DART physics and the simulated IMU is bridged back to `/imu`;

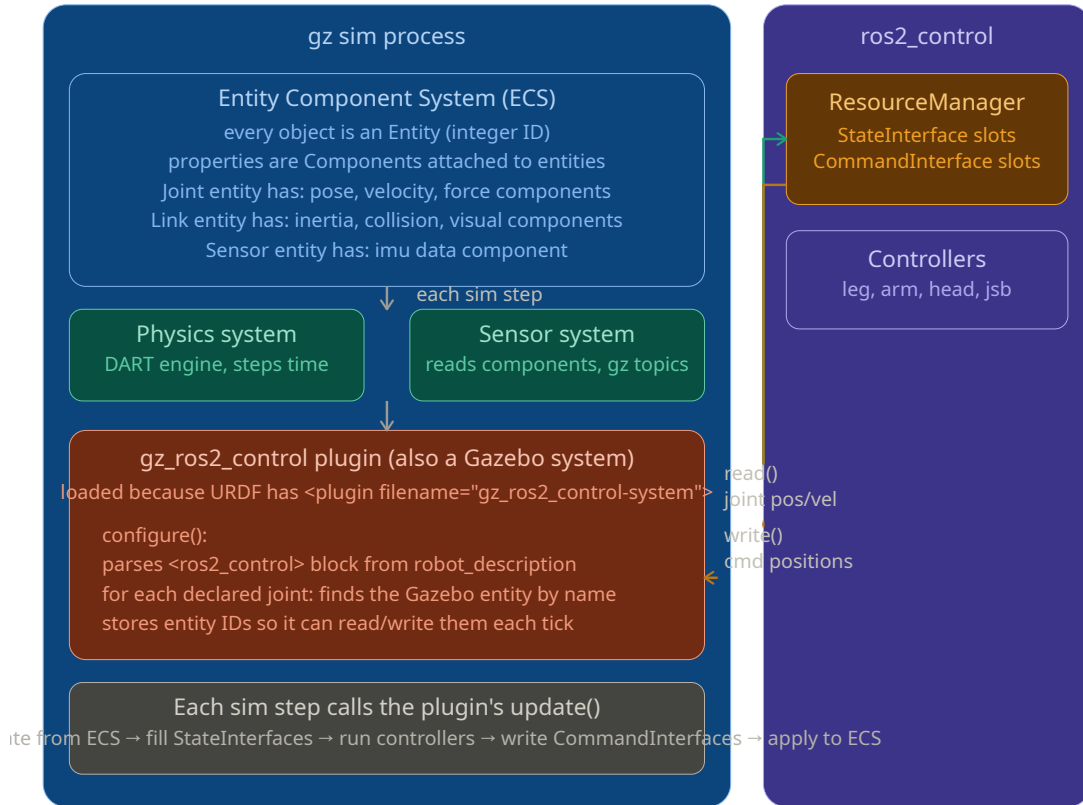


Figure 5.4: How the `gz_ros2_control` plugin operates inside the Gazebo process: it binds the declared joints to simulation entities and exchanges state and commands on every step.

- the **real-robot path**, where the commands are forwarded to the ESP32 over micro-ROS and the MPU6050 publishes `/imu`.

Both paths converge on the same downstream topics — `/imu`, `/joint_states` and the transform tree `/tf` — which the algorithm nodes consume, closing the loop. Because the real robot listens on a single command topic in servo degrees while the controllers publish three topics in radians, a dedicated bridge node converts and re-groups the commands for the firmware; this sim-to-real bridge is what allows the poses validated in simulation to be replayed on the physical robot.

5.8 Cross-Cutting Conventions

A consistent system depends on a few conventions enforced across all packages:

- **Units.** The entire ROS 2 side works in *radians*, following the URDF zero convention; the conversion to servo degrees happens only at the firmware/bridge boundary (Chapter 4), never inside the ROS 2 nodes.
- **Joint ordering.** The order of the joints must match `controllers.yaml` exactly; a single mismatch would silently send a command to the wrong joint.
- **Interfaces.** All message, service and action types live in `humanoid_msgs`, never

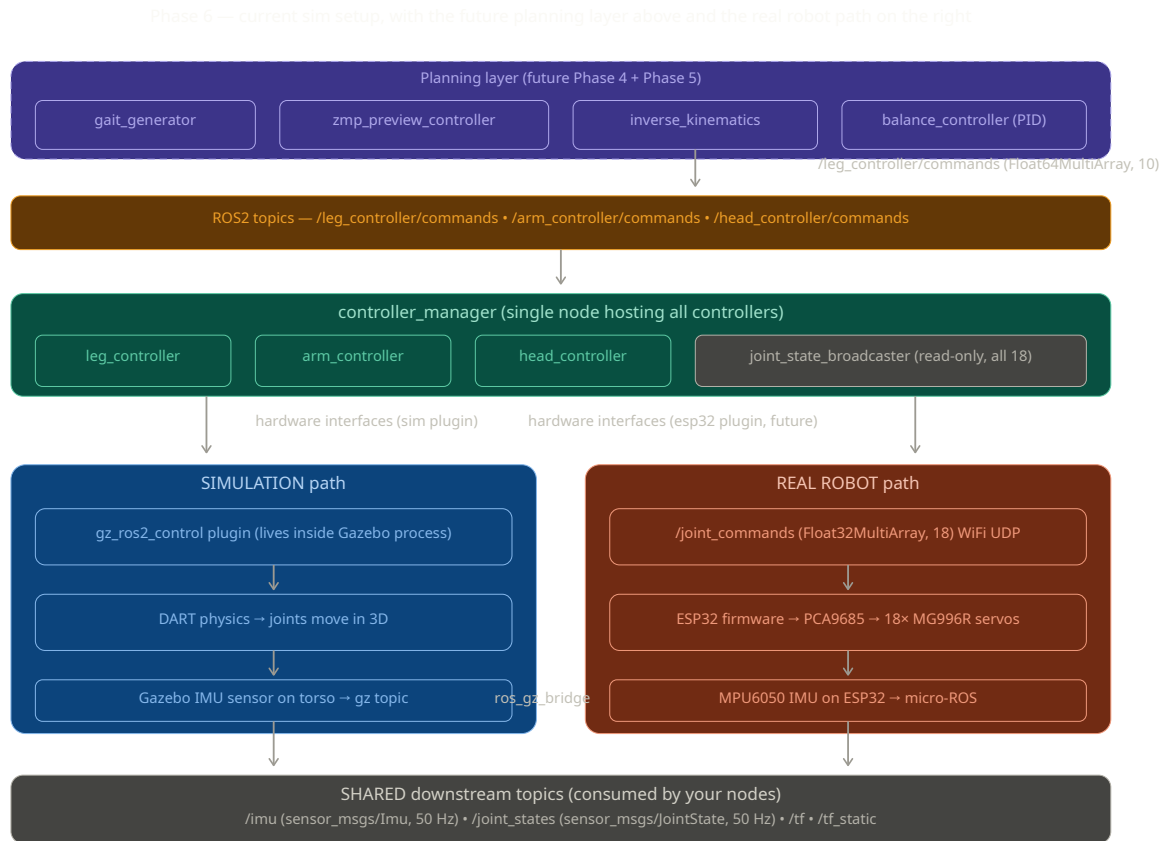


Figure 5.5: The complete node and topic graph: the planning layer feeds the controller manager, which drives either the Gazebo simulation path or the real-robot (ESP32) path; both share the downstream sensor and state topics.

defined ad hoc in an implementation package.

- **Actions vs. topics.** Goal-driven motions use action servers (cancellable, with feedback); streaming data such as the IMU and joint states use topics.

5.9 Conclusion

This chapter presented the software architecture of Abbes: a modular ROS 2 system split into description, bring-up, algorithm, interface and command packages; a robot model that feeds `ros2_control`; a Gazebo digital twin that exposes the very same interfaces as the hardware; and a node graph whose simulation and real-robot paths share a common upstream and a common set of sensor topics. This architecture is what makes the project tractable as software — observable, modular, and portable from simulation to hardware. The next chapter looks inside the algorithm layer, at the kinematics and gait-planning that compute the joint targets flowing through this graph.

Chapter 6

Kinematics and Gait Planning

6.1 Introduction

The software architecture of the previous chapter provides the channels through which joint targets reach the robot. This chapter looks inside the algorithm layer that *computes* those targets. It presents, in turn, the kinematic model of the legs, the inverse kinematics that turns desired foot positions into joint angles, the inertial estimation and balance control that keep the robot upright, and finally the Zero-Moment-Point preview-control pipeline that generates a walking motion. The chapter ends with an honest assessment of how far the resulting walk goes.

6.2 Leg Kinematics

Each leg is a chain of five revolute joints: two at the hip (roll and pitch), one at the knee (pitch), and two at the ankle (pitch and roll). *Forward kinematics* — composing these joint rotations along the thigh and shank links — gives the position of the foot relative to the hip. For locomotion, however, the useful question is the inverse one: given where the foot *should* be, what joint angles place it there? Because the leg has a simple, decoupled structure, this problem admits a closed-form solution that separates cleanly into the sagittal plane (forward/backward motion, Figure 6.1a) and the frontal plane (lateral motion, Figure 6.1b).

6.3 Inverse Kinematics

The robot’s inverse kinematics is implemented as a *closed-form*, two-link geometric solver. An earlier version used an iterative damped-least-squares method, but it proved fragile: depending on its starting guess it could converge to either a slightly-bent or a near-straight leg, so the same foot target produced different poses between runs. The closed-form solver removes this ambiguity — given the hip and foot positions it always returns the same human-like, knee-forward solution — which is essential for a repeatable gait.

The solver works as follows. In the frontal plane, the hip-roll angle is the angle between the vertical and the line from hip to foot (arctan of the lateral over the vertical offset), and the ankle roll is set to its opposite so that the sole stays flat on the ground. In the sagittal plane, the straight-line distance from hip to foot together with the known thigh and shank lengths fixes the knee angle by the *law of cosines*; the hip-pitch angle is the

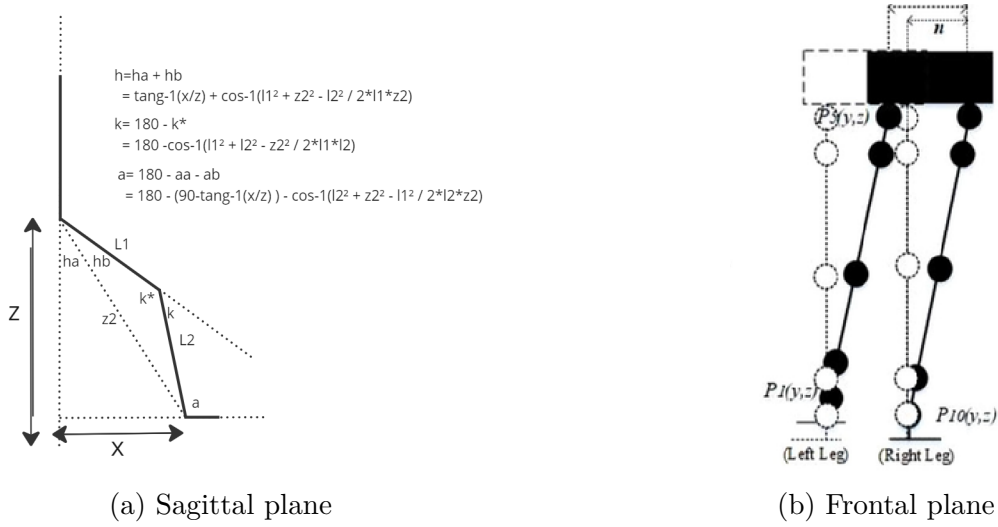


Figure 6.1: Geometric inverse kinematics of the leg, decomposed into the sagittal plane (knee and hip/ankle pitch) and the frontal plane (hip and ankle roll).

direction to the foot minus the offset of the thigh from that line; and the ankle pitch is chosen so that the hip, knee and ankle pitches sum to zero, again keeping the foot flat. Two safeguards protect the solution: the hip-to-foot distance is clamped just below full leg extension, and a minimum knee bend is enforced, both of which keep the leg away from the singular, fully-stretched configuration where the joint angles would be ill-defined. A small forward offset is finally applied to the hip target, because in a knee-bend the true body centre of mass sits slightly ahead of the hip; shifting the target back keeps the centre of mass above the feet.

6.4 Inertial Estimation

Balancing requires a clean estimate of how far the robot is tilted from vertical, but neither inertial signal gives this directly: the accelerometer measures the gravity direction but is corrupted by motion noise, while the gyroscope measures rotation rate cleanly but drifts when integrated over time. A **complementary filter** fuses the two by trusting the integrated gyroscope over the short term and gently correcting it toward the accelerometer estimate over the long term. With its coefficient set to 0.98, the filter follows fast motions smoothly yet remains free of long-term drift. It consumes the raw inertial topic (from the simulated sensor or, on the real robot, from the ESP32) and publishes the filtered pitch and roll used by the balance controller.

6.5 Balance Control

Standing balance is maintained by an inner feedback loop that adjusts the ankle joints in reaction to the measured tilt. Rather than three independent PID gains per axis, the controller uses a **Linear- Quadratic Regulator (LQR)**. The motivation is physical: the ankle couples the pitch and roll axes, so tuning them independently is suboptimal. The LQR instead treats the full state — pitch, roll, and their rates — and computes a single optimal gain matrix offline by solving the algebraic Riccati equation, trading off a

penalty on tilt error against a penalty on ankle effort. At runtime the correction is a single matrix multiplication. Following the same linear-inverted-pendulum model used for gait (Section 6.6), the controller takes the nominal joint angles produced by the kinematics, adds the computed ankle correction, and republishes the corrected command, leaving the rest of the system unchanged.

6.6 Gait Generation by ZMP Preview Control

Walking is generated using the **Zero-Moment-Point preview-control** method of Kajita (2003) [3], built on the Linear Inverted Pendulum Model (LIPM). The LIPM abstracts the whole robot as a point mass at constant height: under this model the ZMP — the point introduced in Chapter 2 — depends on the centre-of-mass position and its acceleration. The walking problem then becomes: find a centre-of-mass trajectory whose resulting ZMP stays inside the support foot at all times. Figure 6.2 shows the relationship between the foot placement, the centre-of-mass path and the ZMP across the single- and double-support phases.

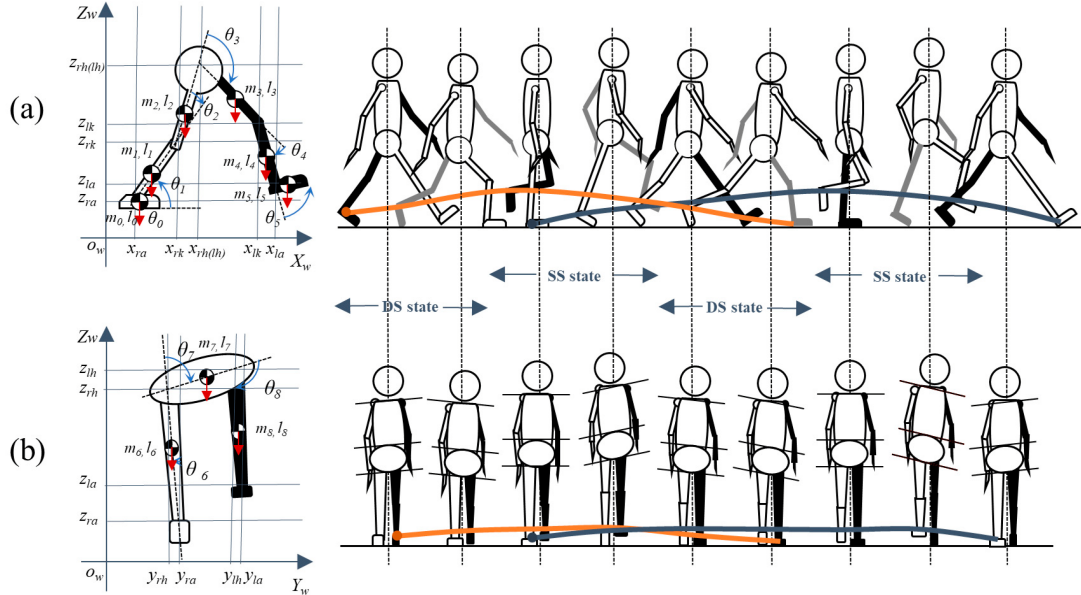


Figure 6.2: Walking in the sagittal (a) and frontal (b) planes: the centre-of-mass trajectory and the ZMP evolve across the double-support (DS) and single-support (SS) phases.

The key difficulty is that the ZMP at the present instant depends on the *future* motion of the centre of mass. The preview controller solves this by looking ahead [7]: it is an optimal regulator, derived from the same Riccati machinery as the balance controller, augmented with a *preview* term that anticipates the reference ZMP over a horizon of about one and a half seconds. This lets the centre of mass start shifting toward the next footstep *before* the foot is placed, exactly as a human leans into a step. The method has since been extended with auxiliary ZMP control and more general preview formulations [5, 6].

The generation is organised as a three-stage pipeline:

1. a **footstep planner** lays out a timed sequence of left/right foot placements along the desired path, with shorter first steps to ramp up from standing;

2. a **ZMP reference** stage converts those footsteps into a target ZMP over time — under the stance foot during single support, interpolating between feet during double support;
3. the **preview controller** turns this ZMP reference into a smooth centre-of-mass trajectory.

In parallel, the swing foot follows a lifted trajectory between its successive placements (Figure 6.3). The centre-of-mass and foot trajectories are then handed to the inverse kinematics of Section 6.3, which produces the joint angles, and the balance controller adds its ankle correction on top. To react to disturbances, the pipeline also closes a *capture-point* loop: from the measured tilt and tilt rate it estimates where the robot would have to step to come to rest, and steers the swing foot’s landing toward that point.

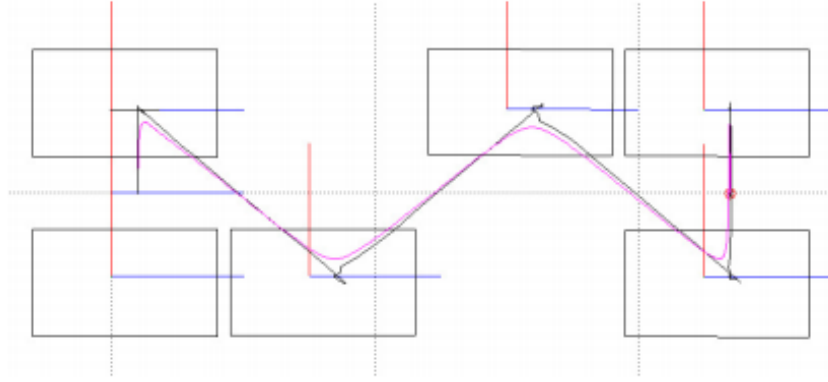


Figure 6.3: Swing-foot trajectory over the planned footstep sequence produced by the walking-pattern generator.

6.7 The Complete Motion Pipeline

Figure 6.4 assembles these components into the full motion pipeline. The feed-forward gait chain (footstep planner → ZMP reference → preview controller) produces the centre-of-mass and foot trajectories; the inverse kinematics converts them to nominal joint angles; and the balance controller adds an ankle correction driven by the inertial feedback. The same inertial feedback feeds the capture-point adjustment, closing the loop back onto the foot placement.

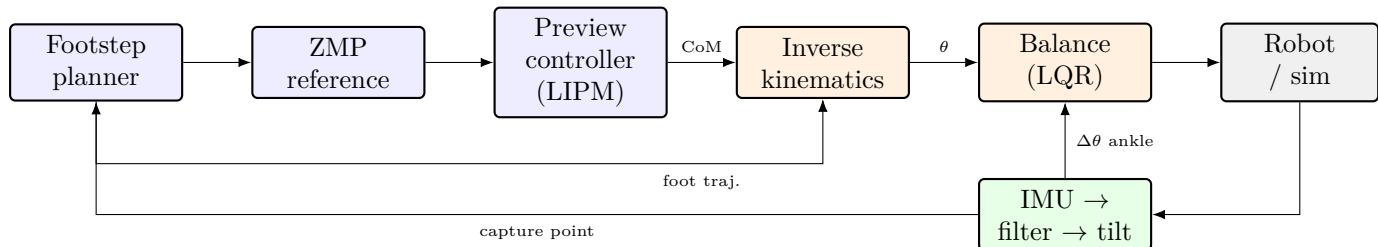


Figure 6.4: The complete motion pipeline: a feed-forward ZMP gait chain feeds the inverse kinematics; the balance controller and the capture-point adjustment close the loop with inertial feedback.

6.8 Results and Limitations

The kinematics and standing balance work well: the closed-form inverse kinematics is deterministic and produces correct, repeatable leg poses, and with the balance controller active the robot holds a deep crouch stably for extended periods in simulation. The gait pipeline produces correct ZMP references, centre-of-mass trajectories and alternating foot motions, and on good runs the robot *marches in place* for several balanced steps.

Robust forward walking, however, remains the open frontier of the project. The fundamental limitation is that the robot is driven by stiff position-controlled hobby servos with no torque sensing, and the lateral balance during single support is only marginally stable: a small side-to-side oscillation tends to grow over several steps. This is a known, method-level limitation rather than a tuning issue — closing it robustly calls for full capture-point step-adjustment feedback or a learned controller (reinforcement learning), which is identified as post-project work in Chapter 9. The detailed experimental results are presented in Chapter 8.

6.9 Conclusion

This chapter presented the algorithms that compute the robot’s joint targets: a closed-form leg inverse kinematics, a complementary filter for inertial estimation, an LQR balance controller, and a Kajita-style ZMP preview-control gait pipeline closed with capture-point feedback. Together they let the robot stand, balance and step in simulation, with robust forward walking remaining the principal open challenge. The next chapter turns to the highest layer of the system: the artificial-intelligence command interface that lets a user drive these motions in natural language.

Chapter 7

The Natural-Language Command Layer

7.1 Introduction

The chapters so far built a robot that can be commanded through ROS2 actions and services. This final implementation chapter adds the highest layer of the system: an interface that lets a person control the robot in ordinary language — “wave your right hand”, “take a few steps back”, “stop!” — and have it act. The layer is built around a locally hosted Large Language Model (LLM) and is the principal artificial-intelligence contribution of the project. This chapter explains its guiding design principle, then walks up the stack: the command vocabulary and its server, the local LLM and tool-use, the dispatcher and its feedback loop, the safety validator, and finally the web interface.

7.2 Design Principle: the LLM Never Moves a Motor

The central design decision is a strict separation of concerns: the language model is allowed to *understand* and to *choose*, but never to *actuate*. It does not generate joint angles or trajectories of any kind. Instead it is given a fixed menu of high-level commands and may only *name* one of them and *fill in its arguments*; a separate, deterministic component is the only thing that turns a name into a real robot motion. This keeps the unpredictable, generative part of the system firmly outside the control loop: whatever the model produces, the robot can only ever execute one of a small set of vetted, bounded motions. Figure 7.1 shows the resulting flow, from the user down to the robot.

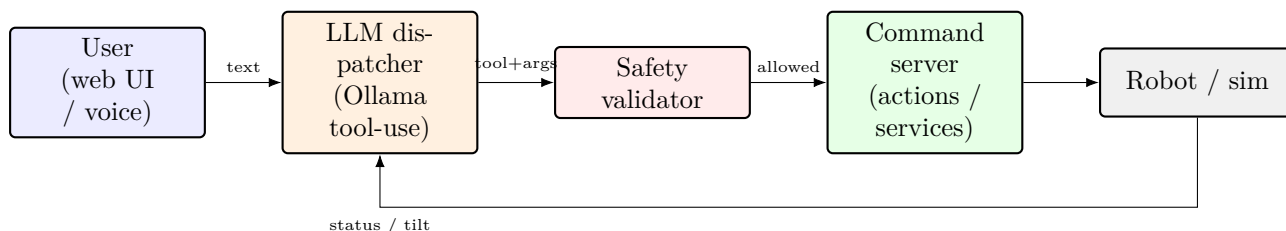


Figure 7.1: The natural-language command layer: the model chooses a command, the safety validator vets it, and only the deterministic command server actuates the robot. Robot status feeds back to inform the next decision.

7.3 The Command Vocabulary and its Server

The robot exposes a fixed vocabulary of seven high-level commands: `walk`, `stand_still`, `sit_down`, `wave_hand`, `bow`, `stop` and `get_status`. These are hosted by a single *command server* node as five ROS 2 **actions** for the goal-driven motions (walking, standing, sitting, waving, bowing) and two **services** for the instantaneous operations (the emergency stop and the status query), using the interfaces defined in `humanoid_msgs` (Chapter 5). The server is the authoritative gatekeeper of motion: it validates each goal’s arguments (for example, a walk distance between 0.1 and 3.0 m), enforces mutual exclusion so that only one motion runs at a time, and checks for cancellation every control cycle so that a stop takes effect within a single tick. Each motion is played as a trajectory that always starts from the robot’s last commanded pose, so commands chain smoothly without snapping. Internally, the server turns each command into the joint trajectories of Chapter 6 and publishes them on the controller topics of Chapter 5.

Crucially, this same vocabulary is described once more as a set of *tool schemas* — one entry per command, listing its name, a plain-language description, and its typed arguments — which is exactly the menu handed to the language model. Because the schemas and the server share one source of truth, what the model is allowed to ask for can never drift from what the robot can actually do.

7.4 The Local Language Model and Tool-Use

The language understanding is performed by a Large Language Model run *locally* through Ollama — the `llama3.2:3b` model, chosen to fit within the 4 GB of the machine’s GPU. Running locally rather than through a cloud service means the system has no external dependency, no per-request cost, and keeps all interaction private and offline, which suits an embedded robotics project.

The mechanism that connects the model to the robot is **tool-use** (also called function-calling). Rather than asking the model to produce free-form text and then trying to parse intentions out of it, the model is given the vocabulary’s tool schemas and is instructed, whenever the user asks for something physical, to respond by *calling* the single tool that best fits and supplying its arguments as structured data. A short system prompt fixes the model’s behaviour: it adopts the identity of “Abbes”, it must call a tool for any physical request and only chat otherwise, it must never invent tools or arguments, and its reply to the user is always one short, friendly, plain-language sentence with none of the underlying machinery exposed. The model thus acts purely as a *translator* from fuzzy human language to a precise, bounded command — the role that language models are genuinely good at, and nothing more.

7.5 The Dispatcher and the Feedback Loop

The *dispatcher* is the deterministic component that mediates between the model and the robot. It mirrors the command server with the matching set of action and service clients, and exposes a single uniform entry point that takes a tool name and arguments, runs the corresponding command to completion, and returns a uniform result (success or failure, with a message).

Rather than a single question-and-answer exchange, the dispatcher runs a bounded

agentic loop (Figure 7.2). It sends the user’s message to the model; if the model calls a tool, the dispatcher executes it on the robot and feeds the *result* back into the conversation as a tool message; the model is then queried again, and the cycle repeats until the model stops calling tools and writes its final reply, or a fixed turn limit is reached. The value of feeding results back is that the model can *react* to what actually happened: if a command is refused or fails, the model sees the real reason and can recover — for instance steadying the robot and retrying — rather than blindly narrating success. The system prompt explicitly instructs the model to use these results to recover before giving up.

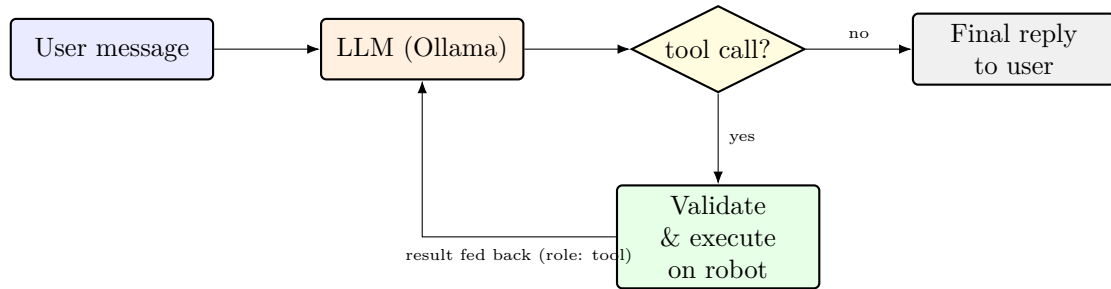


Figure 7.2: The dispatcher’s bounded agentic loop: each tool result is fed back to the model, which can react and retry until it produces a final natural-language reply (subject to a turn limit).

7.6 The Safety Validator

Between the model’s choice and its execution sits a **safety validator** — a thin policy gate consulted after a tool is chosen but before it is run. If the gate denies a command, that command never becomes a robot goal; instead the reason is fed back to the model so the robot can *explain* its refusal rather than silently doing nothing. The validator deliberately does not duplicate the command server’s per-goal argument checks; it adds two rules that depend on physical state and on history rather than on a single message:

- a **tilt gate** that refuses to start `walk` when the robot is already tilted beyond a safe limit, reading the live tilt through the status query;
- a **rate limit** that refuses a new motion if one was allowed too recently, so a chatty model cannot fire a burst of rapid motions.

Two invariants keep the gate safe by construction: `stop` and `get_status` are never blocked — the emergency brake must always fire and a read can never be unsafe — and the gate fails *open* when no inertial data is available, matching the command server’s own non-blocking stance. Because it is pure policy with no robot dependencies, the validator can be unit-tested in isolation.

7.7 The Web Interface

The whole layer is made usable through a browser-based control panel (Figure 7.3). It is served by a deliberately minimal, dependency-free web server built on the Python standard library — no web framework was installed, and the root partition is tight — which embeds the dispatcher in-process and exposes a small set of endpoints: one to

serve the page, one to submit a command, one to poll the robot’s status, and one for the emergency stop. The page is branded around the robot’s identity, “Abbes”, and presents an animated avatar, a live status dashboard with a tilt indicator, clickable cards for the seven commands, and a chat area for free-form requests.

A subtle but important design point concerns concurrency. A motion command blocks while it runs, yet the status poll and the red **stop** button must keep working *during* a motion. The stop and status path is therefore given its own independent ROS 2 context, separate from the dispatcher’s, so that the emergency stop is a genuine interrupt rather than something queued behind the running command.

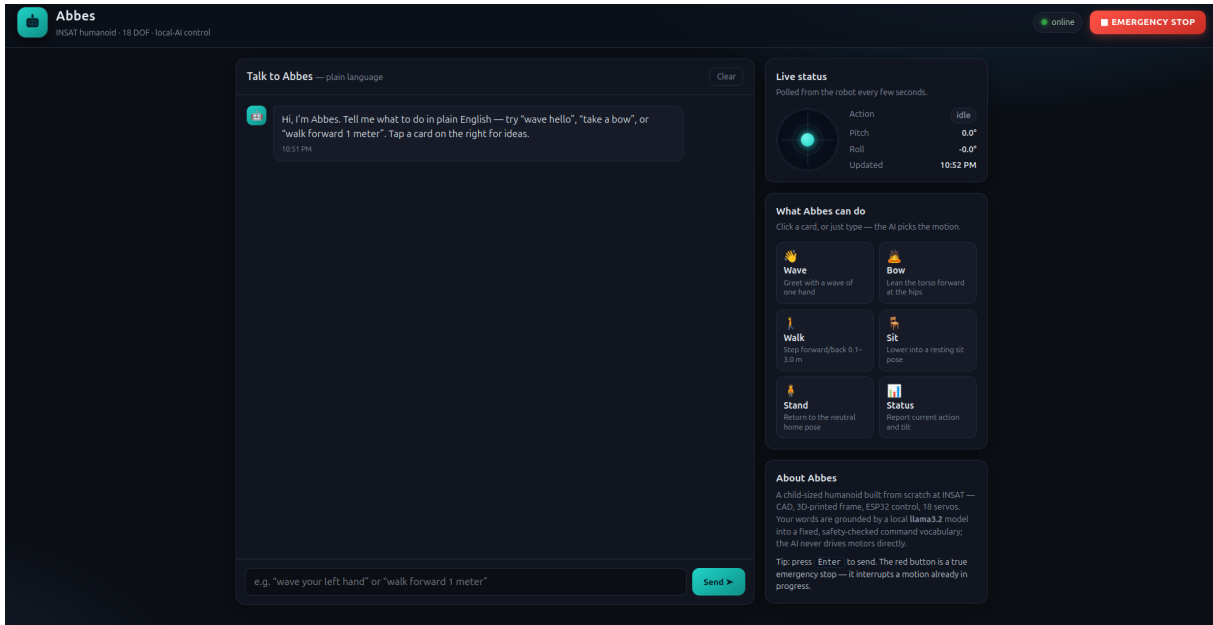


Figure 7.3: The Abbes web control panel: a dependency-free browser interface that drives the natural-language command layer.

7.8 End-to-End Behaviour

Putting the pieces together, a single request flows as follows. The user types a sentence; the dispatcher sends it to the local model with the tool menu; the model replies by calling one tool with its arguments; the safety validator vets that choice against the robot’s tilt and the recent command history; if allowed, the command server plays the corresponding motion to completion; the result is fed back to the model, which produces a short spoken-style confirmation. The same path handles refusals and recovery, and the emergency stop can interrupt any motion in progress.

7.9 Conclusion

This chapter presented the natural-language command layer, the project’s main artificial-intelligence contribution. Its design keeps the language model strictly outside the control loop: the model only translates human language into one of seven vetted commands, a safety validator vets that choice, and a deterministic command server is the only component that actuates the robot. A local LLM, an agentic feedback loop, a safety gate and

a dependency-free web interface together let a person operate the robot conversationally while preserving the safety and determinism that a robot demands. The next chapter evaluates the complete system — motion, balance and this command layer — through simulation and testing.

Chapter 8

Simulation, Testing and Validation

8.1 Introduction

This chapter evaluates the complete system. Because the robot is driven by stiff position servos and a fall in hardware risks the 3D-printed structure, validation is carried out primarily on the Gazebo digital twin introduced in Chapter 5, where behaviour can be observed repeatably and safely. The evaluation proceeds bottom-up, mirroring the order in which the system was built: first standing stability, then active balance, then the walking gait, and finally the natural-language command layer.

8.2 Simulation Environment and Method

The tests run in Gazebo Harmonic with the DART physics engine simulating gravity, contacts and joint dynamics. The simulator can be launched with a graphical view for interactive testing — where the robot can be reset and pushed to probe disturbance rejection — or headless for faster, repeatable runs. The principal quantitative measure throughout is the **torso tilt**: the pitch and roll estimated by the complementary filter from the inertial sensor (Chapter 6). A tilt that stays small indicates an upright, stable robot; a tilt that grows without bound indicates a fall. This single signal is used to judge standing, balancing and walking alike.

8.3 Standing Stability

The first test confirms the foundation: with no controller active, the bare robot is spawned in its geometric standing pose. It stands stably, with the torso tilt holding at roughly half a degree — confirming that the kinematic model, the mass and inertia properties, and the foot contact are all sound, and that any later instability comes from the motion, not from a broken model. Running the full software chain with the gait and balance controllers disabled likewise leaves the robot standing, establishing that the end-to-end pipeline is correct before any active control is added.

8.4 Balance Validation

The second test evaluates active balance on a more demanding pose: a deep knee-bend crouch, which — unlike the rigid straight-leg stand — is an inverted pendulum and is

unstable without feedback. Two ingredients make it hold. First, the target pose is *eased into* over about a second rather than commanded instantly, because the simulated position controller otherwise snaps the joints and the robot lurches before the balancer can react. Second, the LQR ankle controller of Chapter 6 is enabled. With both, the robot holds the crouch stably at around four degrees of tilt for twenty seconds and more — against a fall to tens of degrees within two seconds when balance is off. The controller is effective at holding a settled stance and rejecting gentle pushes; its authority is bounded by the ankle correction limit, so it cannot catch a violent lurch, only maintain a stance that is already near equilibrium.

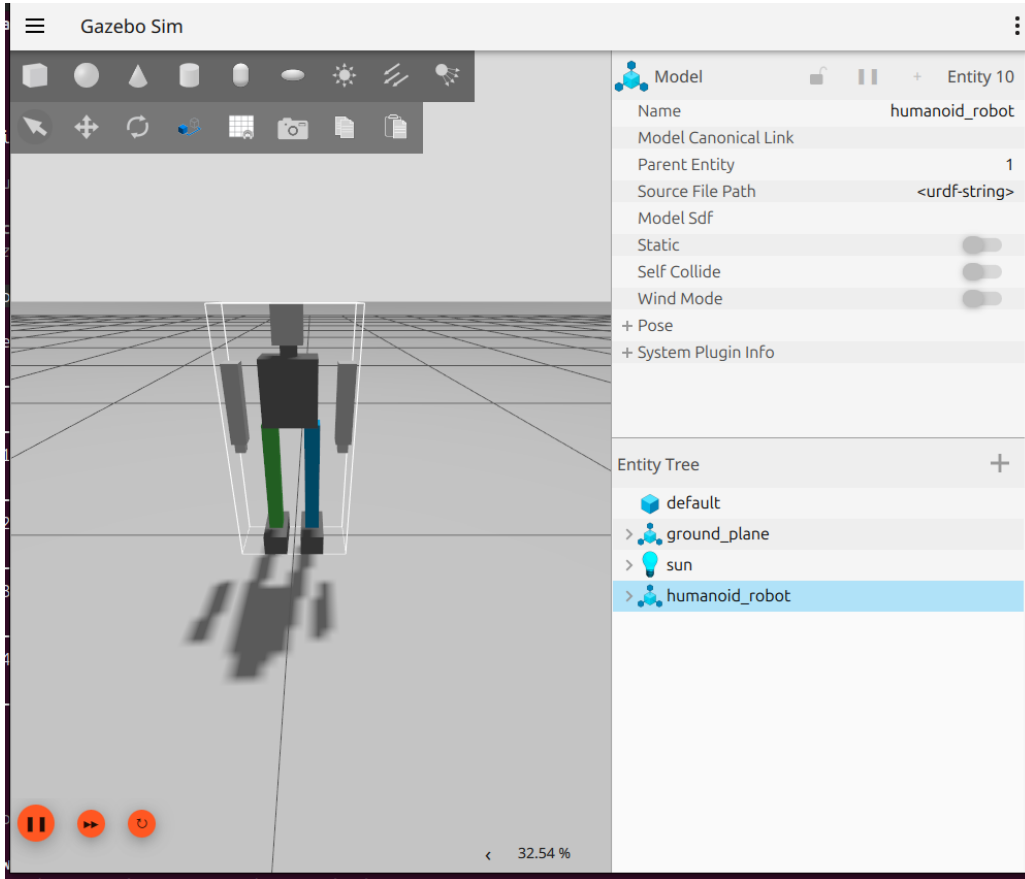


Figure 8.1: The robot standing in the Gazebo simulation, used as the baseline for the balance and gait tests.

8.5 Gait Validation

The third test exercises the full walking pipeline of Chapter 6. The pipeline behaves correctly in its parts: it produces a valid ZMP reference, a smooth centre-of-mass trajectory, and properly alternating foot motions in which a foot only lifts while the centre of mass is over the opposite, supporting foot.

In terms of whole-body behaviour, the honest result is a partial success. On good runs the robot **marches in place** for several balanced steps — on the order of twelve to sixteen seconds — holding a torso tilt in the range of three to seven degrees, and the capture-point step adjustment demonstrably catches near-falls, recovering from tilt spikes that would

otherwise end the run. However, the march is *marginally* stable and varies from run to run on identical code: a slow lateral oscillation tends to build up and eventually tips the robot, and enabling true forward motion tips it sooner. This is not a tuning oversight but a method-level limit: stiff position servos without torque sensing, combined with only marginal lateral balance during single support, place robust dynamic walking beyond the reach of the present ankle-and-hip strategy. Closing the gap calls for full capture-point footstep control or a learned policy, identified as future work in Chapter 9.

8.6 Command-Layer Validation

The natural-language command layer of Chapter 7 was validated end-to-end through the live local model. Plain-English requests are correctly grounded into the right command and executed: waving, bowing, sitting and returning to a standing pose all fire and report back, and the robot’s spoken-style confirmation matches the action performed. The supporting mechanisms were verified individually:

- the **emergency stop** interrupts a motion already in progress — a walk in flight is cancelled and the robot snaps to a safe pose — confirming that the separate-context stop path is a genuine interrupt;
- the **safety validator** was unit-tested across its cases (a level walk is allowed, an immediate second motion is rate-limited, a walk on a tilted base is refused, and `stop` and `get_status` are never blocked);
- the **feedback loop** was verified with a scripted scenario in which a refused command leads the model to steady the robot and then retry successfully, rather than give up.

A noteworthy bug was found and fixed during this validation: under continuous status polling, the stop/status path and the dispatcher initially shared one ROS 2 context and corrupted each other’s internal state. Giving the stop/status path its own context resolved it — an example of the concurrency care that a real interactive robot interface requires. The main residual limitation is the small (3-billion-parameter) model itself, which occasionally mis-formats a tool call or omits an argument; this is a known limitation of small models rather than of the architecture.

8.7 Results Summary

Table 8.1 summarises the validation outcomes.

8.8 Conclusion

The validation confirms a system that is sound from the ground up: a correct physical model, an effective standing balance, a functional gait pipeline that marches in place, and a natural-language command layer that works end-to-end through a local model with safety gating. It also delineates, honestly, the project’s frontier: robust dynamic forward walking on stiff hobby servos remains unsolved and is the natural target of future work. The next chapter draws the general conclusions of the project and lays out those perspectives.

Table 8.1: Summary of validation results.

Capability	Method	Result
Standing stability	Bare robot, no control	Stable ($\sim 0.5^\circ$ tilt)
Static crouch	Closed-form IK + LQR balance + ease-in	Held 20 s+ ($\sim 4^\circ$)
March in place	ZMP preview gait + balance + capture point	12–16 s, $3\text{--}7^\circ$ (good runs)
Forward walking	Gait with forward speed	A few steps, then tips — not robust
NL commands	Local LLM tool-use $\rightarrow$ action	Grounded and executed correctly
Emergency stop	Service on isolated context	Interrupts in-flight motion
Safety validator	Tilt gate + rate limit	Verified (unit + live)
Feedback loop	Agentic recover-then-retry	Verified (scripted)

Chapter 9

General Conclusion and Perspectives

9.1 Summary of the Work

This project set out to design and build, from scratch, a child-sized 18-DOF humanoid robot, **Abbes**, and — with a software-engineering emphasis — to develop the complete software that makes it stand, balance, move and respond to commands. The work spanned the entire stack. The *mechanical* design derived a human-like, eighteen-degree-of-freedom morphology from anthropometric data and realised it in 3D-printed parts. The *electronics* were organised around an ESP32 microcontroller, a PCA9685 PWM driver and an MPU6050 inertial sensor. The *firmware* turned that microcontroller into a real-time, self-reconnecting micro-ROS node, making the on-robot controller a first-class participant in the ROS 2 graph. On top of it, a modular *ROS 2 software architecture* separated the robot model, the simulation bring-up, the algorithms and the interfaces into independent packages, and used `ros2_control` and a Gazebo digital twin so that the same code drives simulation and hardware. The *algorithm* layer contributed a deterministic closed-form leg inverse kinematics, a complementary inertial filter, an LQR balance controller and a ZMP preview-control gait pipeline. Finally, the *artificial-intelligence* layer — the project’s principal contribution — added a natural-language command interface built on a locally hosted language model, in which the model only ever selects from a fixed, safety-checked vocabulary and a deterministic server performs the actual motion.

9.2 Achievements and Limitations

The system works as an integrated whole. The robot stands stably, holds a demanding crouch under active balance, and marches in place through the ZMP gait pipeline in simulation. The command layer operates end-to-end through a local model: spoken-style requests are correctly grounded into robot actions, an emergency stop interrupts any motion, and a safety validator gates unsafe or too-frequent commands. From a software-engineering standpoint, the most valuable outcomes are the architecture itself — a modular, observable, sim-to-real ROS 2 system — and the demonstration that a small, local language model can safely drive a real robot when it is constrained to a vetted command vocabulary.

The project also has clear, honestly acknowledged limitations. Robust dynamic forward walking was not achieved: on the stiff, position-controlled hobby servos, lateral balance during single support is only marginally stable, which is a method-level limitation rather than a tuning one. The current prototype is a single-board build that actuates

sixteen of the eighteen joints, leaving the head unactuated, and it is powered from an off-board source. The language understanding relies on a three-billion- parameter model that occasionally mis-formats a command, a known limit of models at that scale.

9.3 Perspectives

Several directions follow naturally from this work:

- **Robust walking.** The clearest next step is to close the walking gap with a method suited to the hardware: full capture-point footstep control, or a reinforcement-learning policy trained in simulation and transferred to the robot, which is the most promising route for stiff-servo dynamic walking.
- **Voice input.** Adding speech-to-text (for example with a Whisper model) in front of the command layer would let the robot be controlled entirely hands-free, extending the natural-language interface to spoken commands.
- **Graphical path planning.** A graphical interface for picking a target or a path on a map would complement the conversational interface with direct spatial commands.
- **Hardware completion.** A battery-powered version, the re-actuation of the head joints, calibration of the sim-to-real bridge on the physical servos, and a move to torque-capable actuators would together carry the validated simulation behaviour onto robust hardware.
- **Stronger language model.** Swapping in a small model with better function-calling would reduce the occasional command mis-formatting without leaving the local, no-cloud setting.

9.4 Closing Remarks

Beyond the robot itself, the lasting result of this project is a coherent software platform: a humanoid whose every layer, from the microcontroller firmware to the language interface, communicates through one consistent middleware, and whose behaviour can be developed and validated in simulation before reaching hardware. That platform is what makes the remaining challenge — robust locomotion — an incremental next step rather than a new beginning, and it embodies the software-engineering approach that this project set out to apply to robotics.

Bibliography

- [1] M. Shahbazi, R. Babuška, and G. A. M. Lopes, “Unified modeling and control of walking and running on the spring-loaded inverted pendulum,” *IEEE Transactions on Robotics*, 2016, pp. 1178–1195.
- [2] K. Hauser, T. Bretl, K. Harada, and J.-C. Latombe, “Using motion primitives in probabilistic sample-based planning for humanoid robots,” in *Algorithmic Foundation of Robotics VII*, Springer, 2008, pp. 507–522.
- [3] S. Kajita *et al.*, “Biped walking pattern generation by using preview control of zero-moment point,” in *Proc. 2003 IEEE Int. Conf. on Robotics and Automation*, Sept. 2003, pp. 1620–1626.
- [4] F. R. Sias and Y. F. Zheng, “How many degrees-of-freedom does a biped need?,” in *IEEE Int. Workshop on Intelligent Robots and Systems*, 1990, pp. 297–302.
- [5] S. Kajita *et al.*, “Biped walking pattern generation allowing auxiliary ZMP control,” in *Proc. 2006 IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, Oct. 2006, pp. 2993–2999.
- [6] J. Park and Y. Youm, “General ZMP preview control for bipedal walking,” in *IEEE Int. Conf. on Robotics and Automation*, Apr. 2007, pp. 2682–2687.
- [7] T. Katayama, “Design of an optimal controller for a discrete-time system subject to previewable demand,” *International Journal of Control*, vol. 41, no. 3, Mar. 1985, pp. 677–699.